06 / SLIDE 001 Progettazione del Web dinamico e delle API

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 1 di 46

Spiegazione per studiare

La lezione studia come si passa da semplici pagine a servizi Web dinamici. I temi centrali sono progettazione delle API, documentazione OpenAPI, esercizi di definizione di endpoint e organizzazione dell'applicazione in livelli e microservizi.

Definizioni, passaggi ed esempi presenti nella slide

Il Web dinamico

SLIDE ORIGINALE · diagrammi, tabelle e codice

UNICA UNIVERSITÀ DEGLI STUDI DI CAGLIARI

sAIfer Lab
Joint lab on Safety and Security of AI

Il Web dinamico

06 / SLIDE 002 In questa lezione

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 2 di 46

Spiegazione per studiare

Si affrontano prima i principi di progettazione delle API e i contratti di interfaccia; poi si applicano con un esempio concreto in Python. La seconda parte collega tale progettazione alla modularità del codice e alle architetture software di sistemi complessi.

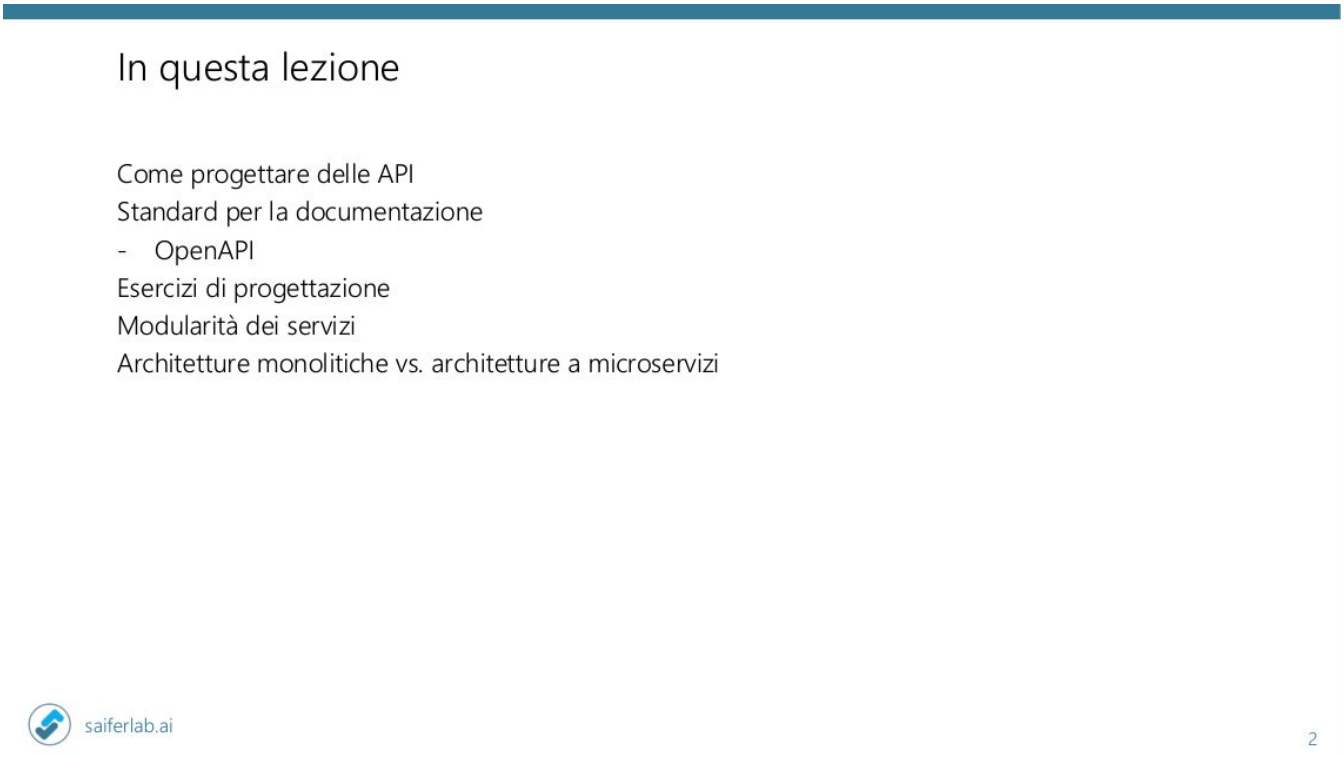
Definizioni, passaggi ed esempi presenti nella slide

Come progettare delle API Standard per la documentazione

- OpenAPI

Esercizi di progettazione Modularità dei servizi Architetture monolitiche vs. architetture a microservizi

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 003 API e servizi

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 3 di 46

Spiegazione per studiare

Il Web si è evoluto da archivio di pagine statiche a insieme di servizi che comunicano attraverso API. La modularità consente di combinare componenti diversi; la trasparenza mantiene nascosti all'utente i dettagli interni del recupero e dell'elaborazione dei dati.

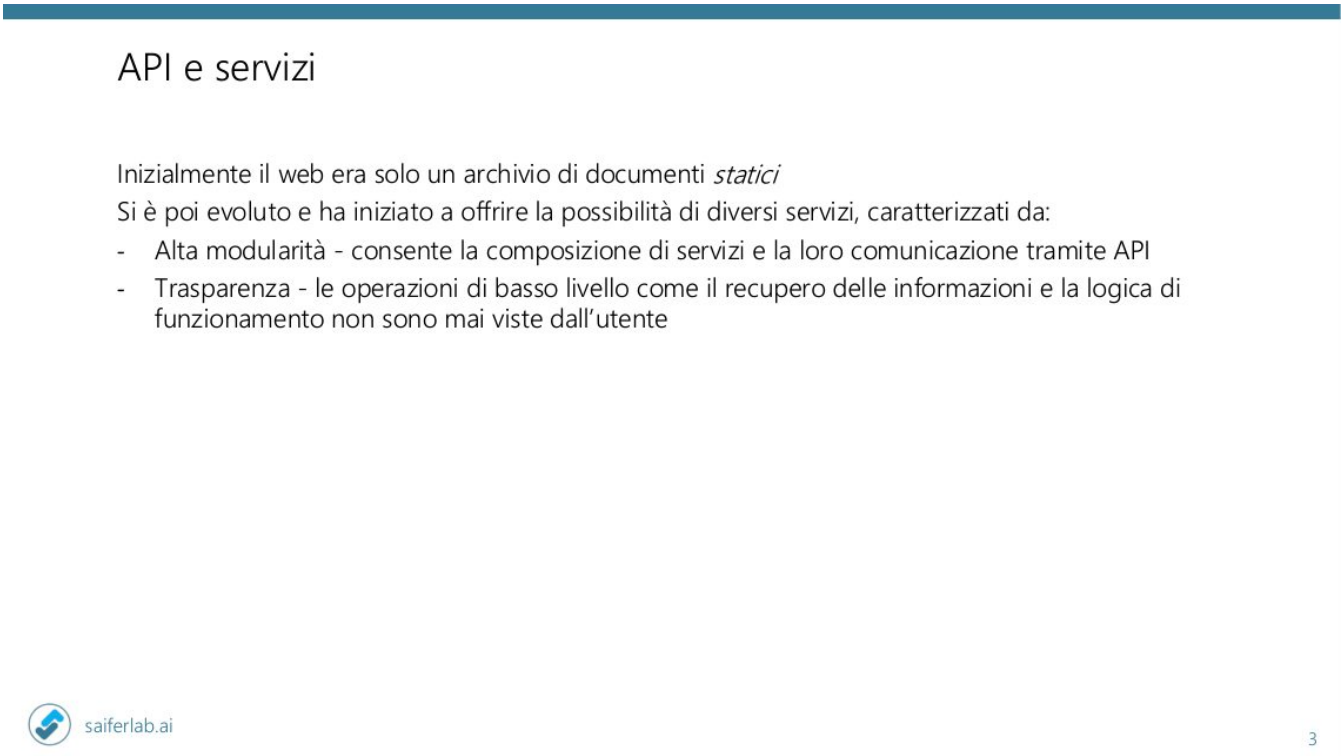
Definizioni, passaggi ed esempi presenti nella slide

Inizialmente il web era solo un archivio di documenti statici Si è poi evoluto e ha iniziato a offrire la possibilità di diversi servizi, caratterizzati da:

• Alta modularità - consente la composizione di servizi e la loro comunicazione tramite API

• Trasparenza - le operazioni di basso livello come il recupero delle informazioni e la logica di funzionamento non sono mai viste dall’utente

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 004 I web server visti dagli sviluppatori

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 4 di 46

Spiegazione per studiare

Dal punto di vista dello sviluppatore si distinguono frontend, che realizza l'interazione con l'utente, e backend, che contiene la logica e le operazioni sui dati. Le API definiscono il punto di contatto tra i due. Il diagramma illustra questa separazione in strati.

Definizioni, passaggi ed esempi presenti nella slide

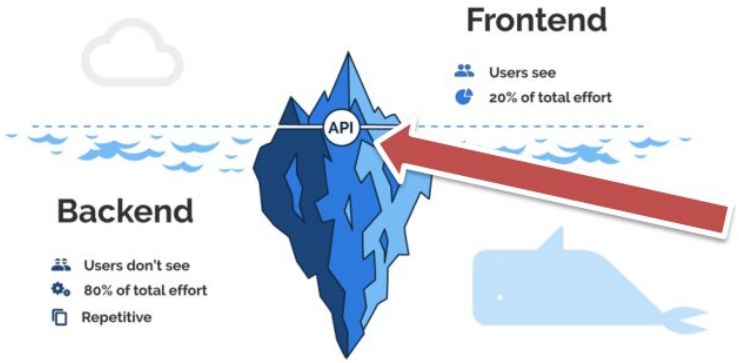
Il server web è una macchina che risponde alle richieste inviate dagli utenti Si divide in frontend (layer di presentazione), e backend (layer di accesso ai dati)

la comunicazione avviene tramite interfacce specifiche (API)

SLIDE ORIGINALE · diagrammi, tabelle e codice

I web server visti dagli sviluppatori

Il server web è una macchina che risponde alle richieste inviate dagli utenti
Si divide in frontend (layer di presentazione), e backend (layer di accesso ai dati)



la comunicazione avviene tramite interfacce specifiche (API)

saiferlab.ai

4

06 / SLIDE 005 Da dove iniziare a creare un web server?

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 5 di 46

Spiegazione per studiare

Si può progettare un servizio partendo dall'interfaccia utente (top-down), dai dati (bottom-up) o dalla logica centrale (middle-out). Nessuna direzione elimina la necessità di pianificare le altre parti: l'interfaccia, il servizio e la persistenza devono poi risultare coerenti.

Definizioni, passaggi ed esempi presenti nella slide

Ci sono diversi approcci

• top-down: partire dall’interfaccia utente

e andare verso le operazioni di basso livello sui dati

• bottom-up: partire dai dati e arrivare

all’interfaccia utente

• middle-out: partire dalla logica di

servizio e lavorare in entrambe le direzioni

Da dove partire molte volte dipende da cosa si ha di pronto Di solito per chi inizia è meglio partire dalla logica, ma SEMPRE iniziare anche a pianificare il resto

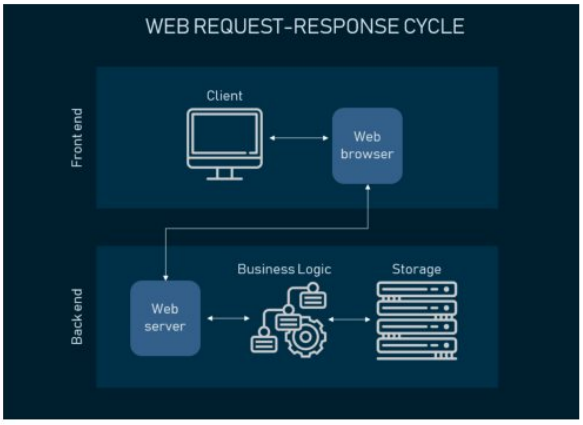
SLIDE ORIGINALE · diagrammi, tabelle e codice

Da dove iniziare a creare un web server?

Ci sono diversi approcci

- **top-down**: partire dall’interfaccia utente e andare verso le operazioni di basso livello sui dati
- **bottom-up**: partire dai dati e arrivare all’interfaccia utente
- **middle-out**: partire dalla logica di servizio e lavorare in entrambe le direzioni

Da dove partire molte volte dipende da cosa si ha di pronto
Di solito per chi inizia è meglio partire dalla logica, ma SEMPRE iniziare anche a pianificare il resto



saiferlab.ai

5

06 / SLIDE 006 Prima di scrivere il codice...

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 6 di 46

Spiegazione per studiare

Una progettazione iniziale chiarisce quali componenti servono e come devono interagire. Scrivere immediatamente il codice rischia di portare a duplicazioni o riprogettazioni costose. La slide usa l'analogia del progetto di una casa per sottolineare questo investimento preliminare.

Definizioni, passaggi ed esempi presenti nella slide

```
La progettazione è una parte
importantissima del processo di sviluppo
```

Se si inizia subito a scrivere il codice, si rischia di perdere tempo a riscrivere pezzi perché non erano progettati bene

```
Il tempo investito in progettazione non è
mai perso! Soprattutto in casi in cui ci sono
delle scadenze temporali strette, è sempre
importante ottimizzare il tempo.
```

Questo si può fare solo tramite una solida progettazione. Immaginate di costruire una casa senza un progetto...

SLIDE ORIGINALE · diagrammi, tabelle e codice

Prima di scrivere il codice...

La progettazione è una parte importantissima del processo di sviluppo

Se si inizia subito a scrivere il codice, si rischia di perdere tempo a riscrivere pezzi perché non erano progettati bene

Il tempo **investito** in progettazione non è mai perso! Soprattutto in casi in cui ci sono delle scadenze temporali strette, è sempre importante ottimizzare il tempo.

Questo si può fare solo tramite una solida progettazione. Immaginate di costruire una casa senza un progetto...



saiferlab.ai

6

06 / SLIDE 007 Come descrivere i servizi e le interazioni

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 7 di 46

Spiegazione per studiare

Progettare un'API significa prevedere quali funzionalità i client richiederanno e renderle accessibili in modo coerente. Non basta definire endpoint isolati: bisogna considerare l'ecosistema delle API, la loro facilità d'uso e le dipendenze tra servizi.

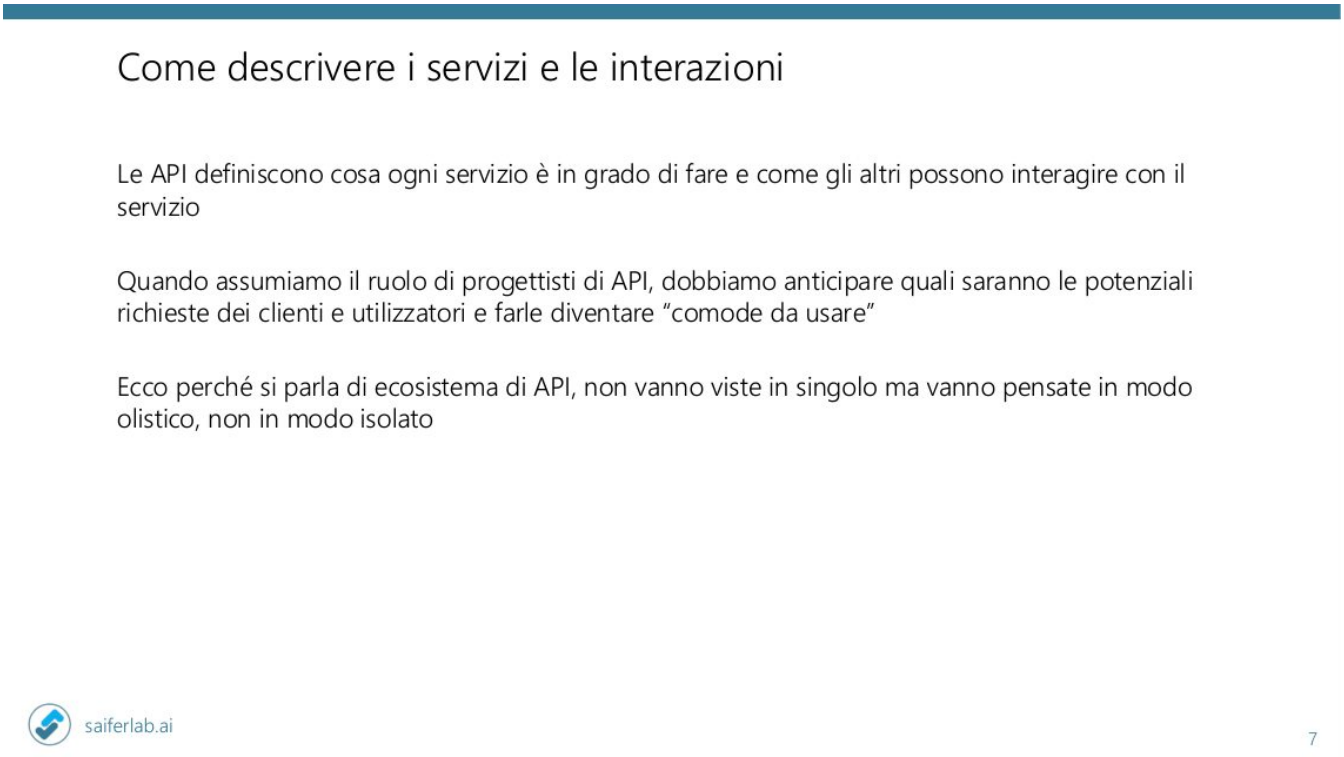
Definizioni, passaggi ed esempi presenti nella slide

Le API definiscono cosa ogni servizio è in grado di fare e come gli altri possono interagire con il servizio

Quando assumiamo il ruolo di progettisti di API, dobbiamo anticipare quali saranno le potenziali richieste dei clienti e utilizzatori e farle diventare “comode da usare”

Ecco perché si parla di ecosistema di API, non vanno viste in singolo ma vanno pensate in modo olistico, non in modo isolato

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 008 Tipi di API

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 8 di 46

Spiegazione per studiare

Un'API specifica un protocollo di comunicazione e un formato dei dati scambiati. La slide cita diverse modalità di interazione tra componenti: invocazioni remote, messaggistica, publish/subscribe e code di lavoro. Ciascuna risponde a differenti esigenze di comunicazione e coordinamento.

Definizioni, passaggi ed esempi presenti nella slide

Ogni API definisce:

- un protocollo, ovvero la struttura di controllo per la comunicazione
- un formato, ovvero la struttura del contenuto

Ci sono poi delle evoluzioni delle iniziali API, che erano pensate solo per comunicazioni dirette

- Remote Procedure Calls (RPC), delle API che chiamano funzioni in altri processi
- Messaging, che mandano piccole porzioni di dati in pipeline, per esempio a seguito di eventi (es., un processo che ha completato l’esecuzione)
- Publish-subscribe (o pub-sub), un protocollo in cui uno dei due lati si iscrive (subscriber) agli aggiornamenti inviati dall’altro (publisher) su un dato argomento
- Code (queues), utilizzate solitamente per gestire più richieste che richiedono tempo per essere eseguite

Tutte queste possono essere utilizzate all’interno di web server per gestire processi più complessi rispetto alla sola visualizzazione di documenti

SLIDE ORIGINALE · diagrammi, tabelle e codice

Tipi di API

Ogni API definisce:
- un **protocollo**, ovvero la struttura di controllo per la comunicazione
- un **formato**, ovvero la struttura del contenuto

Ci sono poi delle evoluzioni delle iniziali API, che erano pensate solo per comunicazioni dirette
- *Remote Procedure Calls (RPC)*, delle API che chiamano funzioni in altri processi
- *Messaging*, che mandano piccole porzioni di dati in pipeline, per esempio a seguito di eventi (es., un processo che ha completato l'esecuzione)
- *Publish-subscribe* (o *pub-sub*), un protocollo in cui uno dei due lati si iscrive (subscriber) agli aggiornamenti inviati dall'altro (publisher) su un dato argomento
- *Code (queues)*, utilizzate solitamente per gestire più richieste che richiedono tempo per essere eseguite

Tutte queste possono essere utilizzate all'interno di web server per gestire processi più complessi rispetto alla sola visualizzazione di documenti

06 / SLIDE 009 Come descrivere le API

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 9 di 46

Spiegazione per studiare

Ogni operazione API può essere descritta tramite input attesi e output prodotti. Documentare tali caratteristiche aiuta sia chi usa il servizio sia chi lo modifica, e facilita la costruzione di test che verifichino il comportamento previsto.

Definizioni, passaggi ed esempi presenti nella slide

Possiamo pensare alle API come delle operazioni sul servizio

Ogni operazione ha un suo input e genera un output

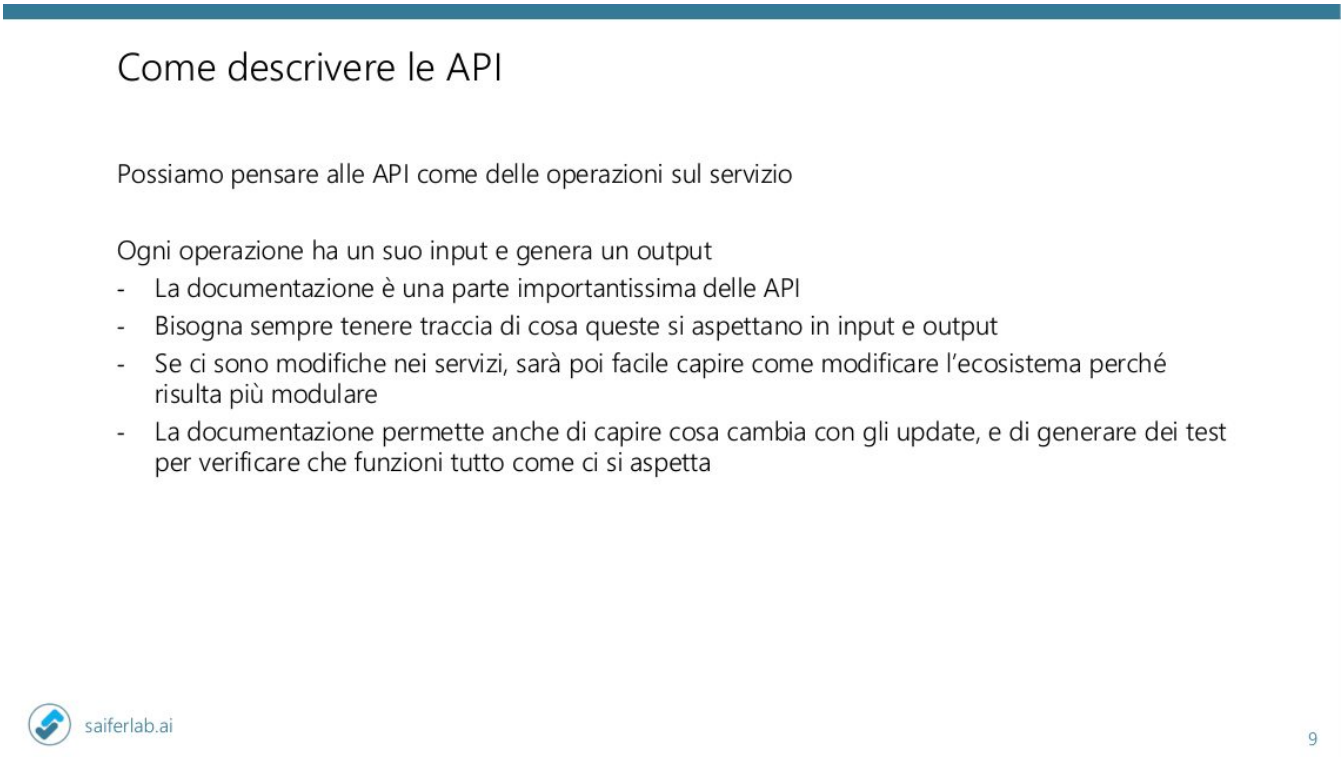
• La documentazione è una parte importantissima delle API

• Bisogna sempre tenere traccia di cosa queste si aspettano in input e output

• Se ci sono modifiche nei servizi, sarà poi facile capire come modificare l'ecosistema perché risulta più modulare

• La documentazione permette anche di capire cosa cambia con gli update, e di generare dei test per verificare che funzioni tutto come ci si aspetta

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 010 Standard per descrivere le API

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 10 di 46

Spiegazione per studiare

OpenAPI è una specifica per documentare interfacce HTTP. Le descrizioni, comunemente in YAML o JSON, possono indicare informazioni generali, operazioni, parametri, richieste, risposte e requisiti di accesso. Il contratto è leggibile sia da sviluppatori sia da strumenti automatici.

Definizioni, passaggi ed esempi presenti nella slide

OpenAPI è una specifica che specifica come descrivere le API HTTP-Based, ovvero quelle che si basano sul protocollo HTTP Tipicamente queste API sono chiamate RESTful (vedremo poi cosa significa)

Il formato OpenAPI descrive in un file di testo gli input e output delle API, insieme anche a informazioni sull'API, per esempio che autorizzazione è richiesta o chi fornisce il servizio

Le definizioni possono essere scritte "a mano" o tramite strumenti (ne vedremo alcuni nel corso)

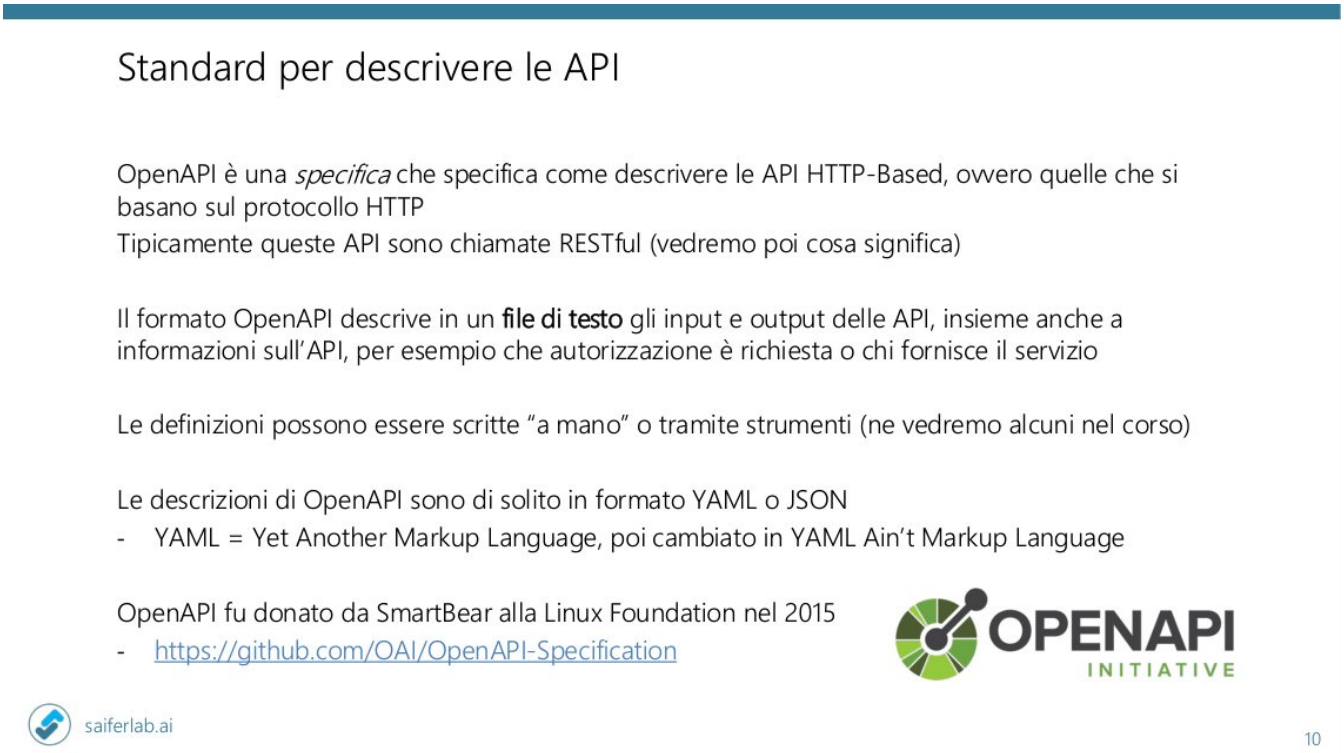
Le descrizioni di OpenAPI sono di solito in formato YAML o JSON

• YAML = Yet Another Markup Language, poi cambiato in YAML Ain't Markup Language

OpenAPI fu donato da SmartBear alla Linux Foundation nel 2015

• https://github.com/OAI/OpenAPI-Specification

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 011 Esempio di definizione OpenAPI

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 11 di 46

Spiegazione per studiare

L'esempio YAML mostra una descrizione OpenAPI con `info`, `servers`, `paths` e la definizione di una richiesta `GET /users`. I blocchi delle risposte dichiarano lo status e il formato dei dati restituiti. La slide cita Swagger Editor per esplorare visivamente il contratto; il testo estratto dal PDF può mescolare le colonne del codice, perciò è utile consultare anche l'immagine della pagina.

Definizioni, passaggi ed esempi presenti nella slide

openapi: 3.0.0 info:

Sembra una descrizione lunga, ma contiene molte informazioni utili sul servizio

```
  title: Sample API
  description: Optional multiline or single-line description
  version: 0.1.9
```

servers:

- url: http://api.example.com/v1

Soprattutto, questo formato è facilmente leggibile da persone (human-readable) e macchine (machine-readable)

description: Optional server description

- url: http://staging-api.example.com

description: Optional server description

paths:

/users:

get:

summary: Returns a list of users. description: Optional extended description in CommonMark or HTML. responses:

Per leggerlo in modo più rapido, possiamo usare un servizio chiamato Swagger

"200": # status code

description: A JSON array of user names content:

application/json:

schema:

type: array items:

https://editor.swagger.io

type: string

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di definizione OpenAPI

Sembra una descrizione lunga, ma contiene molte informazioni utili sul servizio

Soprattutto, questo formato è facilmente leggibile da persone (*human-readable*) e macchine (*machine-readable*)

Per leggerlo in modo più rapido, possiamo usare un servizio chiamato Swagger

https://editor.swagger.io

```yaml
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description
  version: 0.1.9

servers:
  - url: http://api.example.com/v1
    description: Optional server description
  - url: http://staging-api.example.com
    description: Optional server description

paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        "200": # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

06 / SLIDE 012 Perché YAML e non JSON?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 12 di 46

Spiegazione per studiare

JSON e YAML possono rappresentare le stesse informazioni di un contratto OpenAPI. YAML risulta generalmente più compatto e leggibile nei documenti scritti a mano, mentre JSON conserva la struttura di oggetti e array già utilizzata dagli scambi applicativi.

Definizioni, passaggi ed esempi presenti nella slide

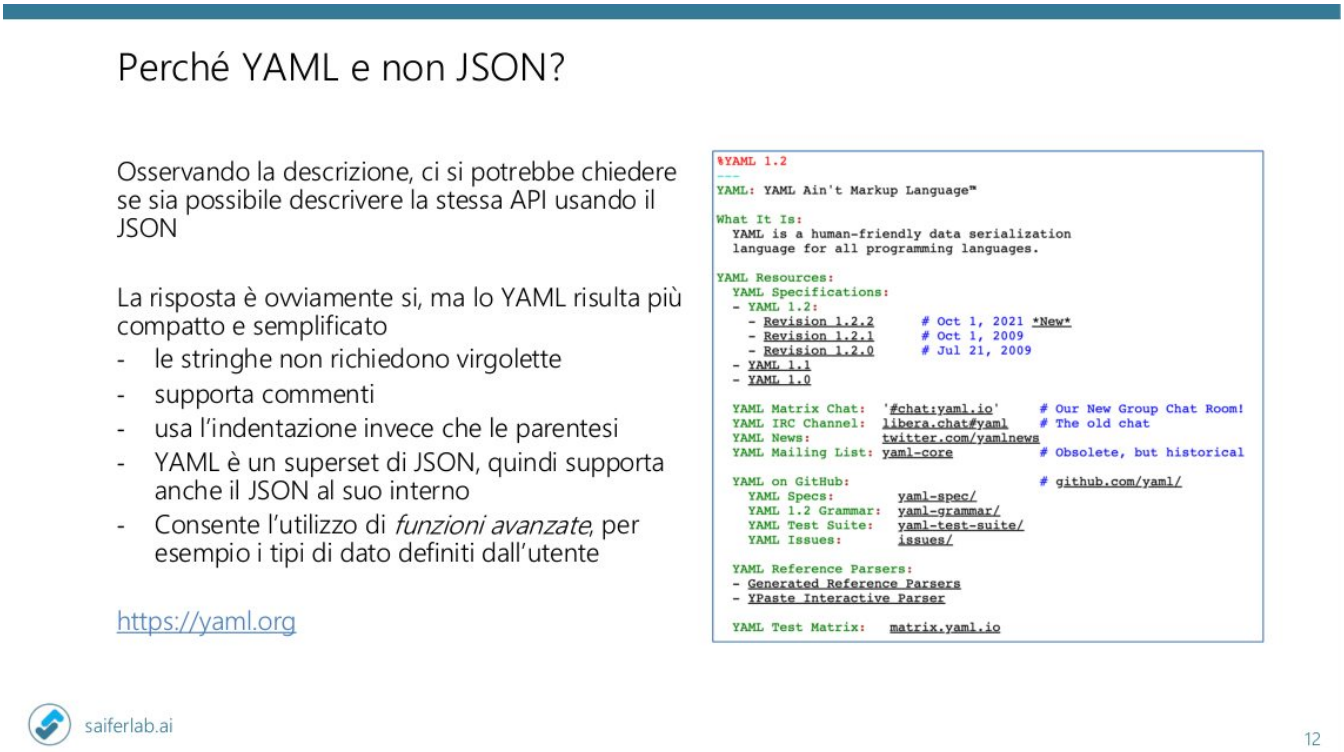
Osservando la descrizione, ci si potrebbe chiedere se sia possibile descrivere la stessa API usando il JSON

La risposta è ovviamente si, ma lo YAML risulta più compatto e semplificato

- le stringhe non richiedono virgolette
- supporta commenti
- usa l'indentazione invece che le parentesi
- YAML è un superset di JSON, quindi supporta anche il JSON al suo interno
- Consente l'utilizzo di funzioni avanzate, per esempio i tipi di dato definiti dall'utente

https://yaml.org

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 013 A cosa servono le definizioni di API?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 13 di 46

Spiegazione per studiare

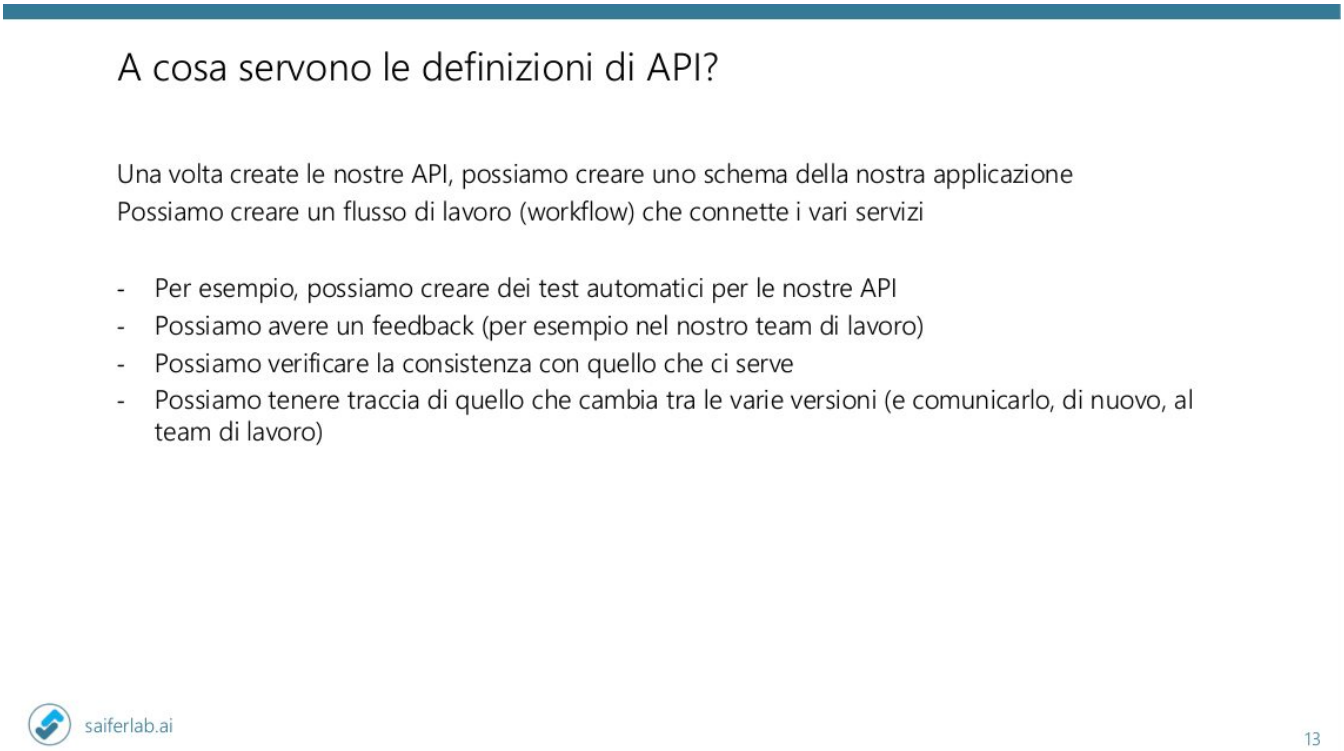
La specifica di un'API non è semplice documentazione descrittiva: può guidare implementazione, verifiche e integrazioni fra componenti. Registrare la forma attesa delle operazioni riduce le ambiguità tra chi sviluppa il server e chi realizza il client.

Definizioni, passaggi ed esempi presenti nella slide

Una volta create le nostre API, possiamo creare uno schema della nostra applicazione Possiamo creare un flusso di lavoro (workflow) che connette i vari servizi

- Per esempio, possiamo creare dei test automatici per le nostre API
- Possiamo avere un feedback (per esempio nel nostro team di lavoro)
- Possiamo verificare la consistenza con quello che ci serve
- Possiamo tenere traccia di quello che cambia tra le varie versioni (e comunicarlo, di nuovo, al team di lavoro)

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 014 Representational State Transfer (REST)

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 14 di 46

Spiegazione per studiare

REST è presentato come uno stile architetturale per servizi basati su risorse e interazioni uniformi. Il punto di studio è capire come risorse identificabili e uso coerente di HTTP consentano di progettare interfacce comprensibili e interoperabili.

Definizioni, passaggi ed esempi presenti nella slide

REST è una collezione di idee su come progettare un sistema interconnesso (in particolare server/client)
Si considera un’architettura software per sistemi distribuiti

Principi base:

- Client-server separation
- Stateless: ogni richiesta è indipendente
- Cacheable: risposte etichettabili come “riutilizzabili”
- Interfaccia uniforme

```
Operazioni CRUD attraverso metodi HTTP per interagire con le risorse:
-
GET (Read)
-
POST (Create)
-
PUT/PATCH (Update)
-
DELETE (Delete)
```

SLIDE ORIGINALE · diagrammi, tabelle e codice

Representational State Transfer (REST)

REST è una collezione di idee su come progettare un sistema interconnesso (in particolare server/client)
Si considera un'architettura software per sistemi distribuiti

Principi base:
- Client-server separation
- Stateless: ogni richiesta è indipendente
- Cacheable: risposte etichettabili come "riutilizzabili"
- Interfaccia uniforme

Operazioni CRUD attraverso metodi HTTP per interagire con le **risorse**:
- GET (Read)
- POST (Create)
- PUT/PATCH (Update)
- DELETE (Delete)



06 / SLIDE 015 Implementazione pratica dei principi REST: le RESTful API

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 15 di 46

Spiegazione per studiare

Una RESTful API mette in pratica i principi REST organizzando operazioni attorno alle risorse e ai metodi HTTP. I client interagiscono attraverso indirizzi e rappresentazioni delle risorse, invece di conoscere le procedure interne del server.

Definizioni, passaggi ed esempi presenti nella slide

“Where REST starts and HTTP ends is a tricky question to answer, but the rule of thumb is that HTTP is the protocol and REST is a way of designing APIs.”

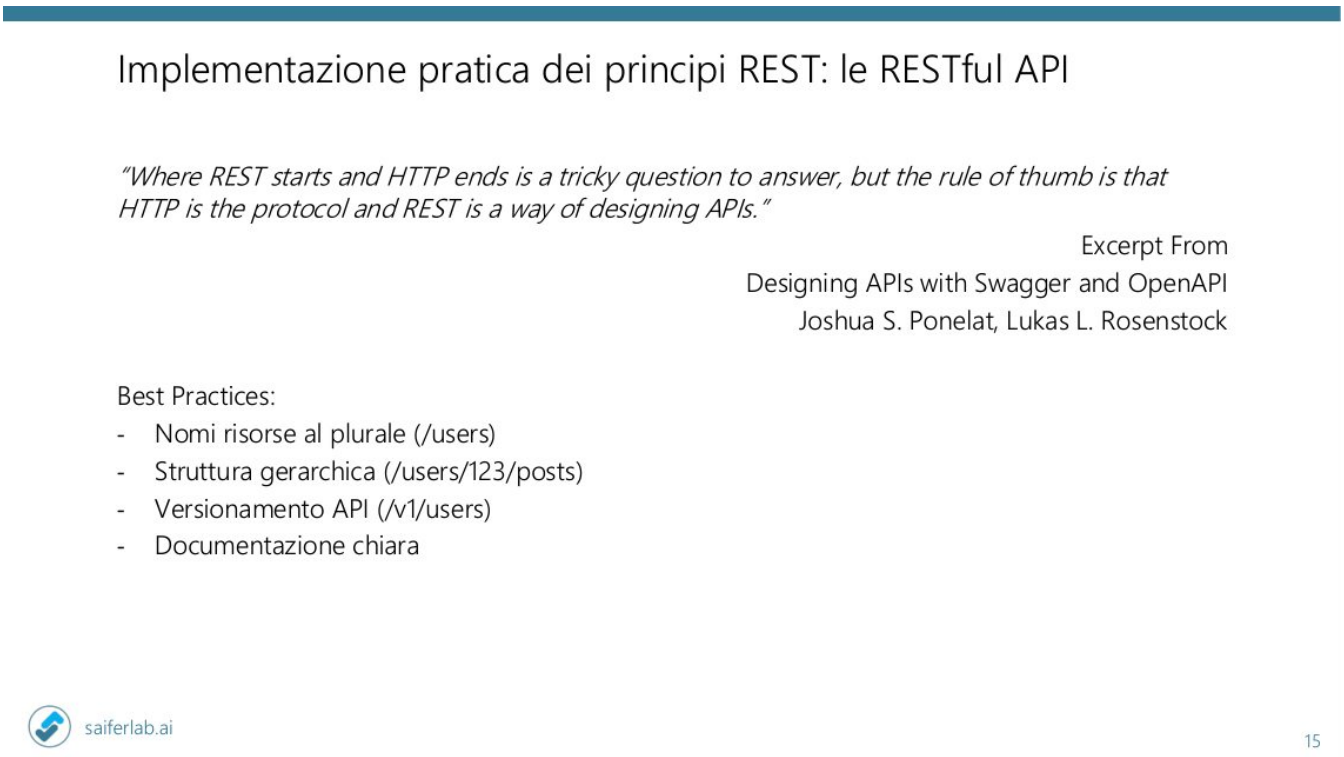
Excerpt From Designing APIs with Swagger and OpenAPI

Joshua S. Ponelat, Lukas L. Rosenstock

Best Practices:

- Nomi risorse al plurale (/users)
- Struttura gerarchica (/users/123/posts)
- Versionamento API (/v1/users)
- Documentazione chiara

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 016 Che utilità ha OpenAPI per gli utenti?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 16 di 46

Spiegazione per studiare

Per chi usa un servizio, una descrizione OpenAPI chiarisce quali operazioni sono disponibili, quali parametri inviare e come interpretare le risposte. Questo rende meno necessario ricavare il comportamento del server soltanto per tentativi.

Definizioni, passaggi ed esempi presenti nella slide

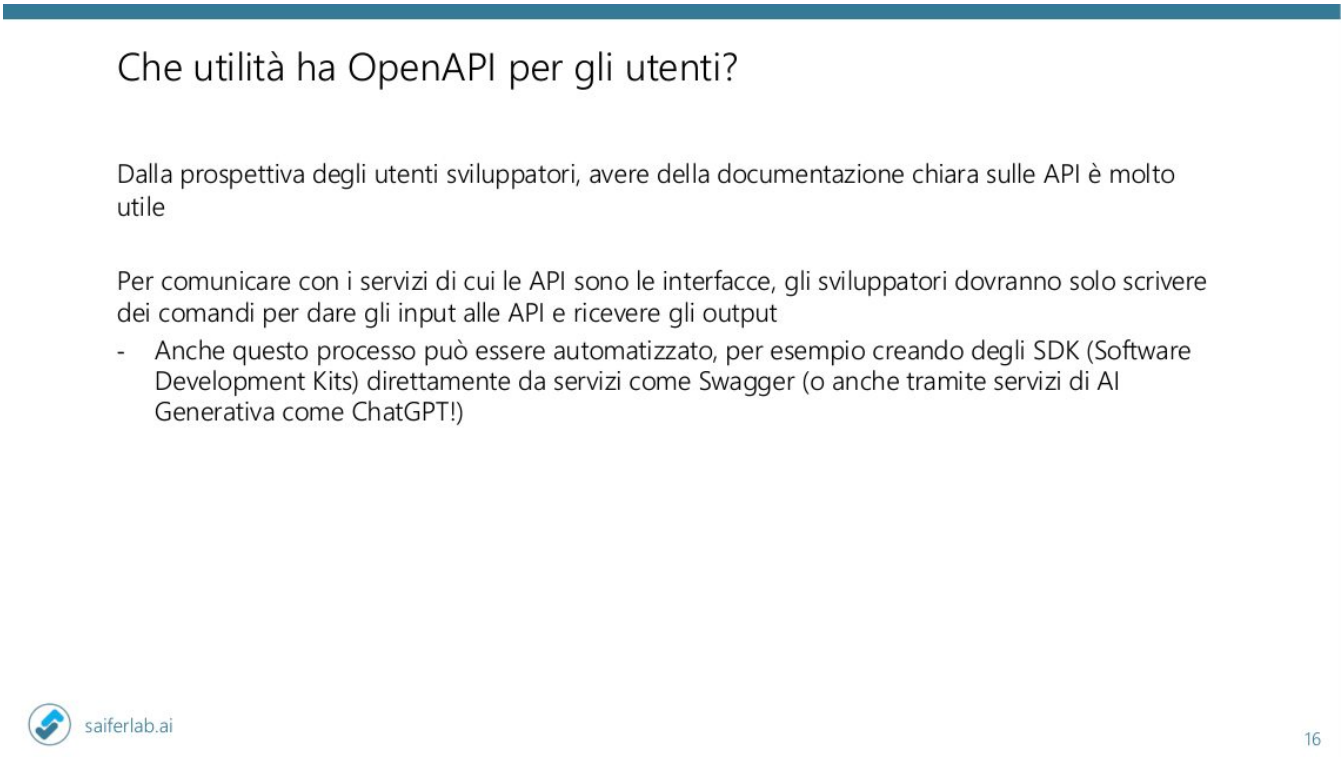
Dalla prospettiva degli utenti sviluppatori, avere della documentazione chiara sulle API è molto utile

Per comunicare con i servizi di cui le API sono le interfacce, gli sviluppatori dovranno solo scrivere dei comandi per dare gli input alle API e ricevere gli output

• Anche questo processo può essere automatizzato, per esempio creando degli SDK (Software

Development Kits) direttamente da servizi come Swagger (o anche tramite servizi di AI Generativa come ChatGPT!)

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 017 Che utilità ha OpenAPI per chi fornisce il servizio?

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 17 di 46

Spiegazione per studiare

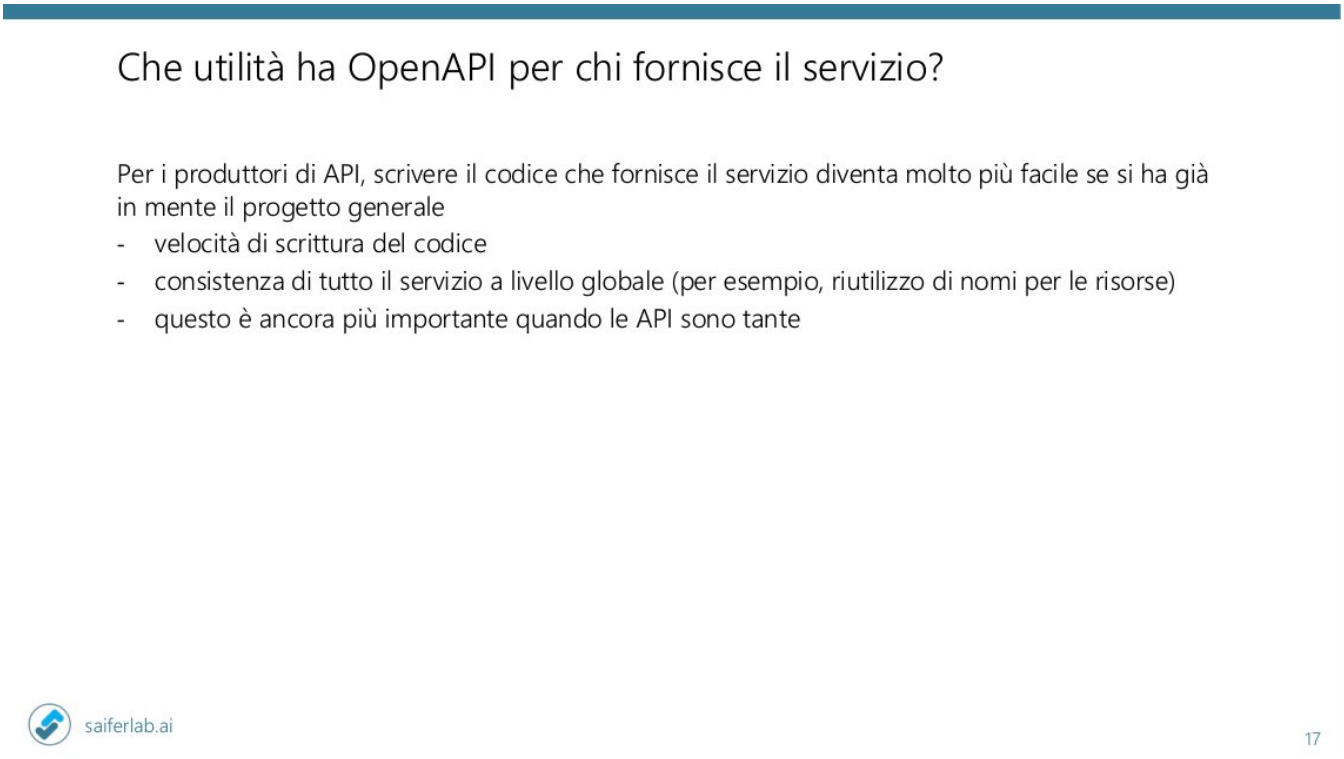
Per chi pubblica il servizio, OpenAPI è utile per mantenere stabile il contratto con i client, supportare test e rendere più comprensibili le modifiche. Una documentazione coerente riduce errori di integrazione quando il backend viene aggiornato.

Definizioni, passaggi ed esempi presenti nella slide

Per i produttori di API, scrivere il codice che fornisce il servizio diventa molto più facile se si ha già in mente il progetto generale

- velocità di scrittura del codice
- consistenza di tutto il servizio a livello globale (per esempio, riutilizzo di nomi per le risorse)
- questo è ancora più importante quando le API sono tante

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 018 Esempio di documentazione di API

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 18 di 46

Spiegazione per studiare

La slide mostra un esempio di documentazione per esplorare un'API. Per studiarla, distinguere indirizzo e metodo HTTP, parametri accettati, formato delle risposte e codici di stato presentati dall'interfaccia.

Definizioni, passaggi ed esempi presenti nella slide

Proviamo, per esempio a navigare queste API

Possiamo usare:

• direttamente il browser

• script, per esempio in Python

• il terminale, tramite

l’applicazione CURL https://curl.se

• strumenti di terze parti (es:

https://www.postman.com)

Documentazione: https://app.beeceptor.com/mock-server/fake-json-api Server: https://fake-json-api.mock.beeceptor.com

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 019 Esercizio in Python

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 19 di 46

Spiegazione per studiare

L'esercizio in Python introduce il passaggio dalla documentazione all'interazione effettiva con un servizio. Il codice della slide va letto individuando quale richiesta viene costruita, quali dati vengono inviati e come viene elaborato il risultato ricevuto.

Definizioni, passaggi ed esempi presenti nella slide

Proviamo a usare una di queste API scrivendo del codice Python

```
import requests

def get_users():

url = ”url_here"
response = requests.get(url)
print("Status code:",
response.status_code)
return response.json()
```

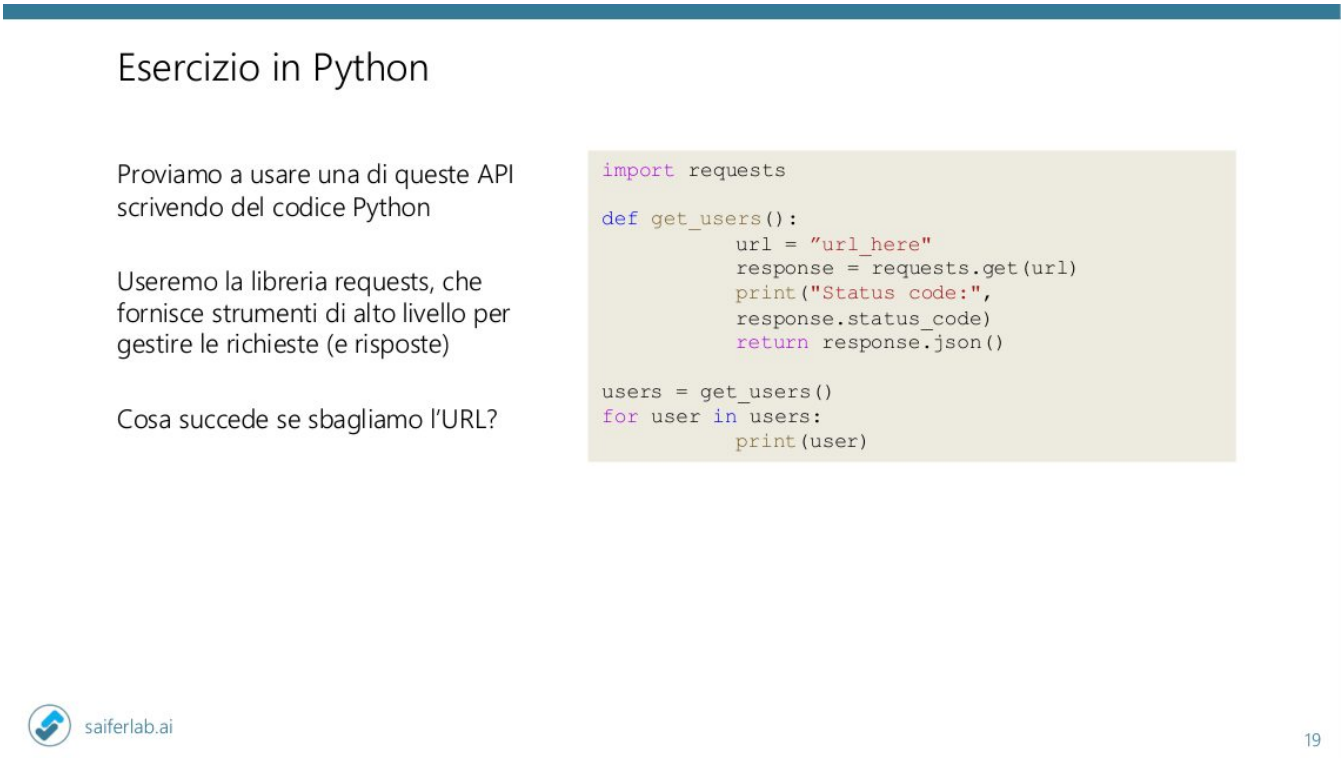
Useremo la libreria requests, che fornisce strumenti di alto livello per gestire le richieste (e risposte)

users = get_users() for user in users:

Cosa succede se sbagliamo l’URL?

print(user)

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 020 Esercizio in Python

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 20 di 46

Spiegazione per studiare

L'esercizio prosegue con un'altra chiamata o con l'elaborazione della risposta. Il punto chiave è osservare che consumare un'API richiede di conoscere il contratto: metodo, indirizzo, parametri e formato della risposta devono essere compatibili.

Definizioni, passaggi ed esempi presenti nella slide

Gestione delle eccezioni

• La gestione deve essere fatta lato

sviluppatori

• Gli utenti non vogliono vedere “errori

rossi” nel loro output

• Nota: Non è una buona pratica

“stampare gli errori”... Per ora serve solo a noi per fare la prova

```
import requests
def get_users():
```

url = ”url_here" try:

```
response = requests.get(url)
response.raise_for_status()
print("Status code:", response.status_code)
return response.json()
except Exception as e:
print("Error:", e)
return None
```

• Potrebbe dare delle informazioni a

potenziali hacker

```
users = get_users()
if users:
```

for user in users:

print(user)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esercizio in Python

Gestione delle eccezioni

- La gestione deve essere fatta lato sviluppatori
- Gli utenti non vogliono vedere “errori rossi” nel loro output
- Nota: Non è una buona pratica “stampare gli errori”... Per ora serve solo a noi per fare la prova
  - Potrebbe dare delle informazioni a potenziali hacker

```
import requests

def get_users():
        url = ”url_here"
    try:
        response = requests.get(url)
        response.raise_for_status()
        print("Status code:", response.status_code)
        return response.json()
    except Exception as e:
        print("Error:", e)
        return None

users = get_users()
if users:
        for user in users:
            print(user)
```

saiferlab.ai

06 / SLIDE 021 E le richieste POST?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 21 di 46

Spiegazione per studiare

Questa pagina invita a osservare come funziona una richiesta POST usando l'API citata nella slide. Il tema è la differenza tra recuperare dati con GET e inviare al server un contenuto da elaborare con POST.

Definizioni, passaggi ed esempi presenti nella slide

Proviamo a capire come funzionano le API di https://funtranslations.com/api/pirate

SLIDE ORIGINALE · diagrammi, tabelle e codice

E le richieste POST?

Proviamo a capire come funzionano le API di https://funtranslations.com/api/pirate

POST /pirate

Translate from English to pirate.

Parameters

Parameter Name	Parameter Type	Description
text	string	Text to translate.

output

The result is a json object with the converted text.

saiferlab.ai

21

06 / SLIDE 022 ... E scriviamo il codice

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 22 di 46

Spiegazione per studiare

Il codice mette in pratica la comunicazione con l'API introdotta in precedenza. Per interpretarlo bisogna seguire la preparazione degli argomenti, l'invio della chiamata e la lettura del risultato. La correttezza dipende dal rispetto delle indicazioni dell'endpoint.

Definizioni, passaggi ed esempi presenti nella slide

url = "https://api.funtranslations.com/translate/pirate.json"

```
def request_pirate_translation(text):
```

try:

```
response = requests.post(url, json={"text": text})
response.raise_for_status()
print("Status code:", response.status_code)
return response.json()
except Exception as e:
print("Error:", e)
return None
text = "Hello and welcome to the Web Development course!"
translation = request_pirate_translation(text)
if translation:
```

print(translation)

SLIDE ORIGINALE · diagrammi, tabelle e codice

... E scriviamo il codice

```
url = "https://api.funtranslations.com/translate/pirate.json"


def request_pirate_translation(text):
      try:
            response = requests.post(url, json={"text": text})
            response.raise_for_status()
            print("Status code:", response.status_code)
            return response.json()
      except Exception as e:
            print("Error:", e)
            return None

text = "Hello and welcome to the Web Development course!"
translation = request_pirate_translation(text)
if translation:
            print(translation)
```

saiferlab.ai

22

06 / SLIDE 023 Ora di scrivere la nostra prima API

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 23 di 46

Spiegazione per studiare

La lezione passa dalla chiamata a API esistenti alla progettazione di un'interfaccia propria. Prima di programmare conviene descrivere i dati e le operazioni attese, così che il comportamento del servizio sia specificato indipendentemente dai dettagli implementativi.

Definizioni, passaggi ed esempi presenti nella slide

Vogliamo definire un servizio di biblioteca in cui possiamo listare dei libri e mettere delle recensioni
Iniziamo a pensare a cosa potrà fare l'utente

- Ottenere la lista di libri
- Ottenere il dettaglio di un singolo libro
- Inserire una recensione per il libro scelto

Quindi ci serviranno tre API (almeno per l'inizio) La seconda è gerarchicamente legata alla prima

- Possiamo pensare a una struttura gerarchica delle risorse

Inoltre, mentre le prime due sono API di richiesta informazioni, la terza è un inserimento

- Le prime due saranno GET, la terza sarà una POST

SLIDE ORIGINALE · diagrammi, tabelle e codice

Ora di scrivere la nostra prima API

Vogliamo definire un servizio di biblioteca in cui possiamo listare dei libri e mettere delle recensioni
Iniziamo a pensare a cosa potrà fare l'utente

- Ottenere la lista di libri
- Ottenere il dettaglio di un singolo libro
- Inserire una recensione per il libro scelto



Quindi ci serviranno tre API (almeno per l'inizio)
La seconda è gerarchicamente legata alla prima
- Possiamo pensare a una struttura **gerarchica** delle risorse

Inoltre, mentre le prime due sono API di richiesta informazioni, la terza è un inserimento
- Le prime due saranno **GET**, la terza sarà una **POST**

saiferlab.ai

23

06 / SLIDE 024 Ripasso: Metodi HTTP più usati

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 24 di 46

Spiegazione per studiare

La slide riprende i verbi HTTP principali e il loro significato nelle API. Il ripasso serve a scegliere, per ogni operazione del servizio, il metodo più adatto invece di far transitare tutte le azioni attraverso una sola tipologia di richiesta.

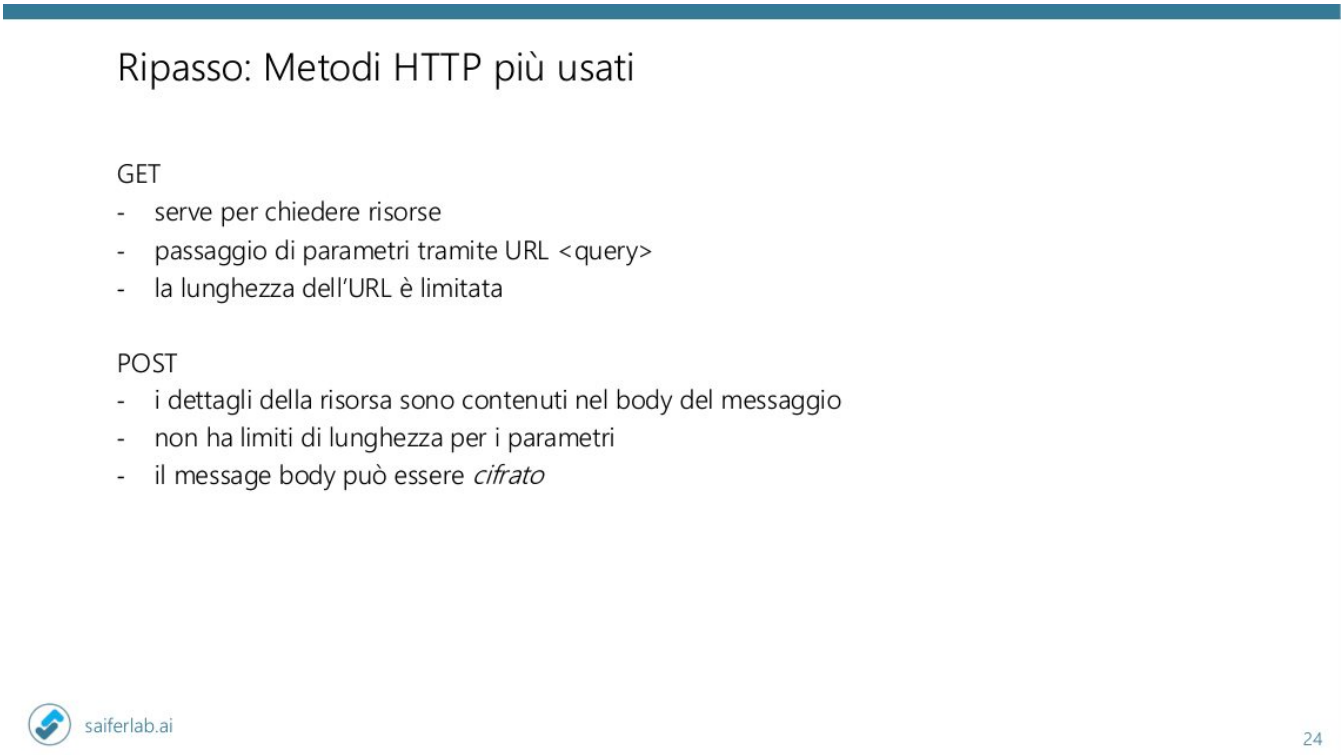
Definizioni, passaggi ed esempi presenti nella slide

```
GET
-
serve per chiedere risorse
-
passaggio di parametri tramite URL <query>
-
la lunghezza dell’URL è limitata
```

POST

- i dettagli della risorsa sono contenuti nel body del messaggio
- non ha limiti di lunghezza per i parametri
- il message body può essere cifrato

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 025 Ripasso: Status code HTTP

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 25 di 46

Spiegazione per studiare

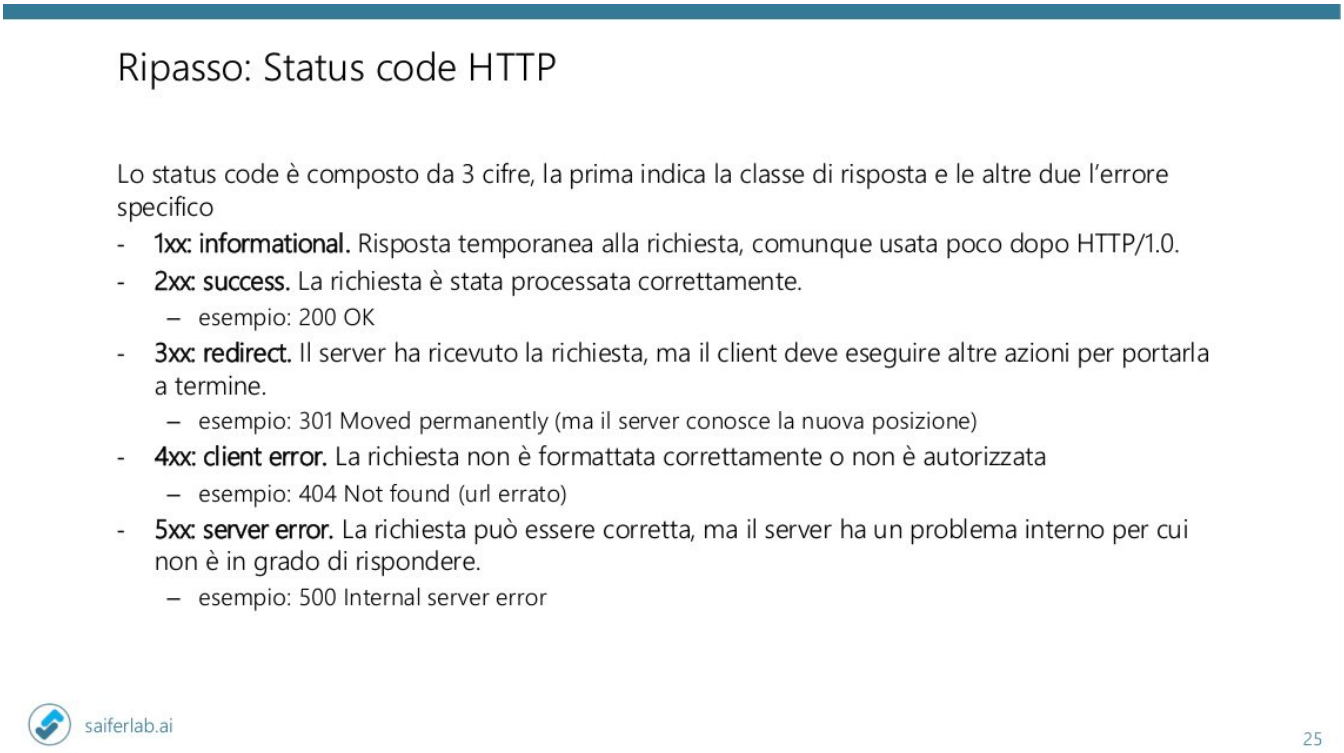
Anche gli status code devono essere previsti nella definizione dell'interfaccia. Un endpoint ben descritto specifica non solo la risposta di successo, ma anche i casi di input errato, risorsa non trovata e problema interno al servizio.

Definizioni, passaggi ed esempi presenti nella slide

Lo status code è composto da 3 cifre, la prima indica la classe di risposta e le altre due l'errore specifico

- 1xx: informational. Risposta temporanea alla richiesta, comunque usata poco dopo HTTP/1.0.
- 2xx: success. La richiesta è stata processata correttamente.
- esempio: 200 OK
- 3xx: redirect. Il server ha ricevuto la richiesta, ma il client deve eseguire altre azioni per portarla a termine.
- esempio: 301 Moved permanently (ma il server conosce la nuova posizione)
- 4xx: client error. La richiesta non è formattata correttamente o non è autorizzata
- esempio: 404 Not found (url errato)
- 5xx: server error. La richiesta può essere corretta, ma il server ha un problema interno per cui non è in grado di rispondere.
- esempio: 500 Internal server error

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 026 Definiamo prima il formato di un libro

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 26 di 46

Spiegazione per studiare

L'esempio progettuale sceglie il libro come entità applicativa. Prima degli endpoint si definisce quali campi caratterizzano un libro e quali valori il client e il server dovranno scambiarsi. Questo modello condiviso dà coerenza alle operazioni successive.

Definizioni, passaggi ed esempi presenti nella slide

Per ogni libro, in questo caso d’uso specifico, ci serviranno tre attributi

- Titolo
- Autore
- Recensione

Che tipo di dato usare?

- Per titolo e autore possiamo pensare a delle stringhe
- Per la recensione possiamo scegliere una recensione

testuale, ma per ora utilizziamo un numero da 1 a 5

SLIDE ORIGINALE · diagrammi, tabelle e codice

Definiamo prima il formato di un libro

Per ogni libro, in questo caso d’uso specifico, ci serviranno tre attributi

- Titolo
- Autore
- Recensione

Che tipo di dato usare?

- Per titolo e autore possiamo pensare a delle stringhe
- Per la recensione possiamo scegliere una recensione testuale, ma per ora utilizziamo un numero da 1 a 5



saiferlab.ai

26

06 / SLIDE 027 Formato del libro

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 27 di 46

Spiegazione per studiare

La descrizione del formato del libro rende esplicita la struttura dell'oggetto scambiato tramite le API. È importante distinguere i tipi dei singoli attributi, eventuali identificatori e i dati obbligatori da quelli accessori.

Definizioni, passaggi ed esempi presenti nella slide

Iniziamo a definire che attributi dovrà avere ogni libro Questo ci consente di avere un oggetto riutilizzabile e non riscrivere ogni volta tutta la struttura del libro

- si può vedere come un tipo di dato astratto
- si possono mettere dei constraint su che

forma avranno i dati (es. recensione numero intero da 1 a 5)

components:

schemas:

Libro:

type: object properties:

id:

type: integer example: 1 titolo:

type: string example: "Il nome della rosa" autore:

type: string example: "Umberto Eco" recensione:

type: integer minimum: 1 maximum: 5 example: 5

SLIDE ORIGINALE · diagrammi, tabelle e codice

Formato del libro

Iniziamo a definire che attributi dovrà avere ogni libro

Questo ci consente di avere un oggetto riutilizzabile e non riscrivere ogni volta tutta la struttura del libro

- si può vedere come un tipo di dato astratto
- si possono mettere dei constraint su che forma avranno i dati (es. recensione numero intero da 1 a 5)

```yaml
components:
 schemas:
  Libro:
   type: object
   properties:
    id:
     type: integer
     example: 1
    titolo:
     type: string
     example: "Il nome della rosa"
    autore:
     type: string
     example: "Umberto Eco"
    recensione:
     type: integer
     minimum: 1
     maximum: 5
     example: 5
```



saiferlab.ai

27

06 / SLIDE 028 Passiamo alla prima API: la lista di libri

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 28 di 46

Spiegazione per studiare

La prima API si occupa della collezione di libri. Il percorso rappresenta una risorsa di insieme e l'operazione descrive come ottenerne la lista. La specifica della risposta è fondamentale per stabilire se verrà restituito un array di oggetti e con quale struttura.

Definizioni, passaggi ed esempi presenti nella slide

Iniziamo con l’API per recuperare la lista di libri

paths:

```
-
La richiesta avvenuta con successo risponderà
200 e restituirà un JSON con la lista dei libri
-
Ogni elemento della lista è di tipo libro, come
definito sopra in “components”
-
Definiamo anche un caso di errore in cui il
server risponderà 500 (es. la lista di libri è
troppo grande per essere gestita)
```

/books:

get:

summary: Recupera la lista dei libri

description: Restituisce un elenco di libri.

responses:

'200’:

description: Lista recuperata.

content:

application/json:

schema:

type: array

items:

$ref: '#/components/schemas/Libro’ '500’:

description: Errore interno del server

SLIDE ORIGINALE · diagrammi, tabelle e codice

Passiamo alla prima API: la lista di libri

Iniziamo con l'API per recuperare la lista di libri

- La richiesta avvenuta con successo risponderà 200 e restituirà un JSON con la lista dei libri
- Ogni elemento della lista è di tipo libro, come definito sopra in "components"
- Definiamo anche un caso di errore in cui il server risponderà 500 (es. la lista di libri è troppo grande per essere gestita)

```
paths:
  /books:
    get:
      summary: Recupera la lista dei libri
      description: Restituisce un elenco di libri.
      responses:
        '200':
          description: Lista recuperata.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Libro'
        '500':
          description: Errore interno del server
```

06 / SLIDE 029 Come identificare un singolo libro?

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 29 di 46

Spiegazione per studiare

La seconda operazione deve individuare un libro specifico anziché l'intera raccolta. Un identificatore inserito nel percorso è il meccanismo descritto per selezionare la risorsa. Occorre chiarire come si comporta il server quando il libro non esiste.

Definizioni, passaggi ed esempi presenti nella slide

Abbiamo due scelte: possiamo usare i parametri di path o i parametri di query

• path: solitamente usati per identificare una singola risorsa (come un libro!)

• query: solitamente usati per filtrare o riordinare i risultati (per esempio di una query su un database)

Nel nostro caso il path parameter sembra fare al caso nostro Possiamo vederlo come il numero civico di un indirizzo (l'URL è un Locator alla fine)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Come identificare un singolo libro?

Abbiamo due scelte: possiamo usare i parametri di path o i parametri di query
- path: solitamente usati per identificare una singola risorsa (come un libro!)
- query: solitamente usati per filtrare o riordinare i risultati (per esempio di una query su un database)

Nel nostro caso il path parameter sembra fare al caso nostro
Possiamo vederlo come il numero civico di un indirizzo (l'URL è un Locator alla fine)

Feature	Path Variable	Query Parameter
Location	Part of the URL	Attached to the end of the URL
Purpose	Identify a specific resource	Filter or sort results, or provide additional info

saiferlab.ai

29

06 / SLIDE 030 Recupero di un singolo libro — /books/{id}

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 30 di 46

Spiegazione per studiare

L'operazione di recupero del singolo libro collega parametro di percorso, tipo del risultato e risposte possibili. L'esempio serve a capire la differenza fra documentare un endpoint generico e specificare le singole condizioni della sua esecuzione.

Definizioni, passaggi ed esempi presenti nella slide

summary: Recupera un libro specifico

Per prendere un singolo libro, visto che abbiamo definito i libri come delle risorse, possiamo specificare un ID di libro come richiesta gerarchica annidata tramite path parameter

description: Restituisce i dettagli di un libro specifico dato il suo ID.

parameters:

• name: id

in: path

required: true description: ID del libro da recuperare

schema:

type: integer responses:

```
Definiamo sotto il path /books un altro
path con /books/{id}
```

'200’:

description: Dettagli del libro recuperati con successo content:

Anche questa API restituisce un libro, per cui riusiamo quanto definito prima

application/json:

schema: $ref: '#/components/schemas/Libro’

'404’:

In questo caso dobbiamo anche prenderci cura di possibili ID inesistenti (errore 404: not found)

description: Libro non trovato

'500’:

description: Errore interno del server

SLIDE ORIGINALE · diagrammi, tabelle e codice

Recupero di un singolo libro

Per prendere un singolo libro, visto che abbiamo definito i libri come delle risorse, possiamo specificare un ID di libro come richiesta gerarchica annidata tramite **path parameter**

Definiamo sotto il path /books un altro path con /books/{id}

Anche questa API restituisce un libro, per cui riusiamo quanto definito prima

In questo caso dobbiamo anche prenderci cura di possibili ID inesistenti (errore 404: not found)

```
/books/{id}:
 get:
  summary: Recupera un libro specifico
  description: Restituisce i dettagli di un libro specifico          dato il suo ID.
  parameters:
    - name: id
      in: path
      required: true
      description: ID del libro da recuperare
      schema:
        type: integer
  responses:
   '200':
    description: Dettagli del libro recuperati con successo
    content:
      application/json:
        schema:
         $ref: '#/components/schemas/Libro'
   '404':
     description: Libro non trovato
   '500':
     description: Errore interno del server
```

06 / SLIDE 031 Infine, la richiesta POST per la recensione

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 31 di 46

Spiegazione per studiare

L'operazione POST per una recensione rappresenta un'azione che invia dati al servizio. Si deve descrivere il formato dell'input e l'esito previsto. A differenza di una semplice GET, la richiesta porta al server informazioni da elaborare secondo la logica applicativa.

Definizioni, passaggi ed esempi presenti nella slide

Nel caso della richiesta POST, dobbiamo definire anche il formato del body della richiesta

/books/{id}/recensione:

post:

summary: Aggiungi una recensione a un libro description: Permette di aggiungere una recensione a un libro dato il suo ID. parameters:

• name: id

in: path required: true description: ID del libro a cui aggiungere la recensione schema:

Inoltre, dobbiamo gestire anche casi in cui la richiesta è malformata (per esempio se l’utente inserisce un numero maggiore di 5 o una stringa)

type: integer requestBody:

required: true content:

application/json: schema:

type: object properties:

recensione:

type: integer

minimum: 1 maximum: 5 example: 5 responses:

'200’:

description: Recensione aggiunta con successo '400’:

description: Richiesta non valida '404’:

description: Libro non trovato '500’:

description: Errore interno del server

SLIDE ORIGINALE · diagrammi, tabelle e codice

Infine, la richiesta POST per la recensione

Nel caso della richiesta POST, dobbiamo definire anche il formato del body della richiesta

Inoltre, dobbiamo gestire anche casi in cui la richiesta è malformata (per esempio se l'utente inserisce un numero maggiore di 5 o una stringa)

```yaml
/books/{id}/recensione:
  post:
    summary: Aggiungi una recensione a un libro
    description: Permette di aggiungere una recensione a un libro dato il suo ID.
    parameters:
      - name: id
        in: path
        required: true
        description: ID del libro a cui aggiungere la recensione
        schema:
          type: integer
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              recensione:
                type: integer
                                minimum: 1
                                maximum: 5
                                example: 5
    responses:
      '200':
        description: Recensione aggiunta con successo
      '400':
        description: Richiesta non valida
      '404':
        description: Libro non trovato
      '500':
        description: Errore interno del server
```

06 / SLIDE 032 Ora che la nostra API è definita...

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 32 di 46

Spiegazione per studiare

Terminata la definizione dell'interfaccia, si può passare all'implementazione. Il contratto preparato nelle slide precedenti è il riferimento per scrivere il codice del server e controllarne il comportamento.

Definizioni, passaggi ed esempi presenti nella slide

Possiamo provare a ispezionare la documentazione generata da Swagger https://editor.swagger.io

Ovviamente la logica non è ancora stata scritta, per cui non è possibile provare effettivamente le API

Per scrivere la logica si userà il framework FastAPI

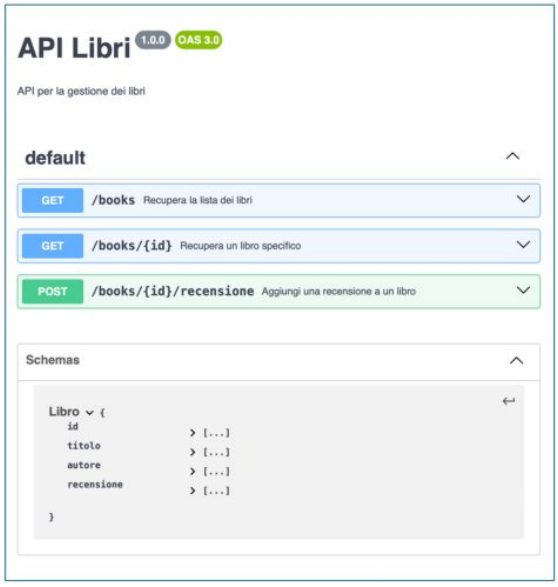
SLIDE ORIGINALE · diagrammi, tabelle e codice

Ora che la nostra API è definita...

Possiamo provare a ispezionare la documentazione generata da Swagger
https://editor.swagger.io

Ovviamente la logica non è ancora stata scritta, per cui non è possibile provare effettivamente le API

Per scrivere la logica si userà il framework FastAPI



saiferlab.ai

32

06 / SLIDE 033 Perché tutto questo lavoro?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 33 di 46

Spiegazione per studiare

L'esercitazione mette in evidenza il valore della progettazione prima del codice: descrivere le API significa prendere decisioni esplicite, individuare le risorse e stabilire che cosa deve restituire ogni operazione. Queste decisioni guidano lo sviluppo e i test.

Definizioni, passaggi ed esempi presenti nella slide

OpenAPI è una specifica formale per descrivere API basate su HTTP, usando il formato JSON o YAML

Una descrizione formale può essere più facilmente interpretata da un software, mentre una informale no

• Cosa significa in pratica? Per esempio, si può

generare del codice già “pronto” con tutte le interfacce specificate (mock server)

• Anche scrivere i test diventa più facile perché si

conosce il formato della risposta

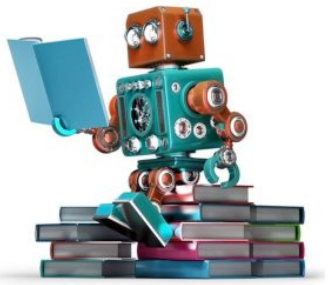
SLIDE ORIGINALE · diagrammi, tabelle e codice

Perché tutto questo lavoro?

OpenAPI è una **specifica formale** per descrivere API basate su HTTP, usando il formato JSON o YAML

Una descrizione formale può essere più facilmente interpretata da un software, mentre una informale no

- Cosa significa in pratica? Per esempio, si può generare del codice già “pronto” con tutte le interfacce specificate (mock server)
- Anche scrivere i test diventa più facile perché si conosce il formato della risposta



saiferlab.ai

33

06 / SLIDE 034 Il passaggio inverso: dal codice alla documentazione

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 34 di 46

Spiegazione per studiare

Oltre al percorso che parte dalla documentazione e arriva al codice, esiste il passaggio inverso: usare strumenti per ottenere o mostrare una descrizione dell'API a partire dall'applicazione. È utile quando l'ambiente di sviluppo integra documentazione e definizione degli endpoint.

Definizioni, passaggi ed esempi presenti nella slide

Nel corso useremo FastAPI, che consente il passaggio inverso: si inizia a scrivere il codice e la documentazione sarà generata (e aggiornata) automaticamente

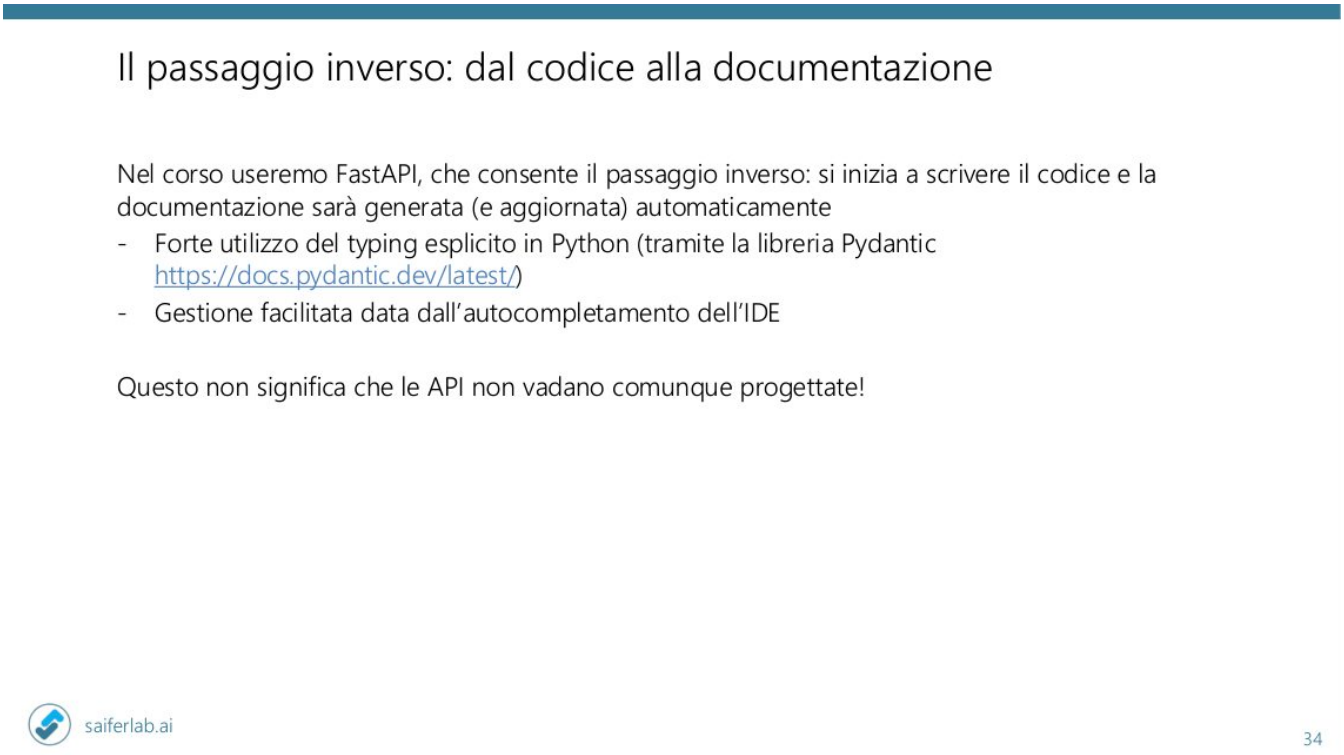
• Forte utilizzo del typing esplicito in Python (tramite la libreria Pydantic

https://docs.pydantic.dev/latest/)

• Gestione facilitata data dall'autocompletamento dell'IDE

Questo non significa che le API non vadano comunque progettate!

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 035 Iniziamo a pensare a dei possibili test...

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 35 di 46

Spiegazione per studiare

Una volta implementati gli endpoint, la specifica aiuta a costruire casi di test. È opportuno verificare sia le richieste attese sia gli scenari di errore, confrontando le risposte del server con il contratto definito.

Definizioni, passaggi ed esempi presenti nella slide

Una volta implementate le nostre API, dovremo fare dei test per verificare che si comportino come pianificato in fase di progettazione

- risposta corretta in caso di successo
- risposta corretta in caso di errori
- per esempio, cosa potrebbe generare errori nella specifica appena definita?

type: integer

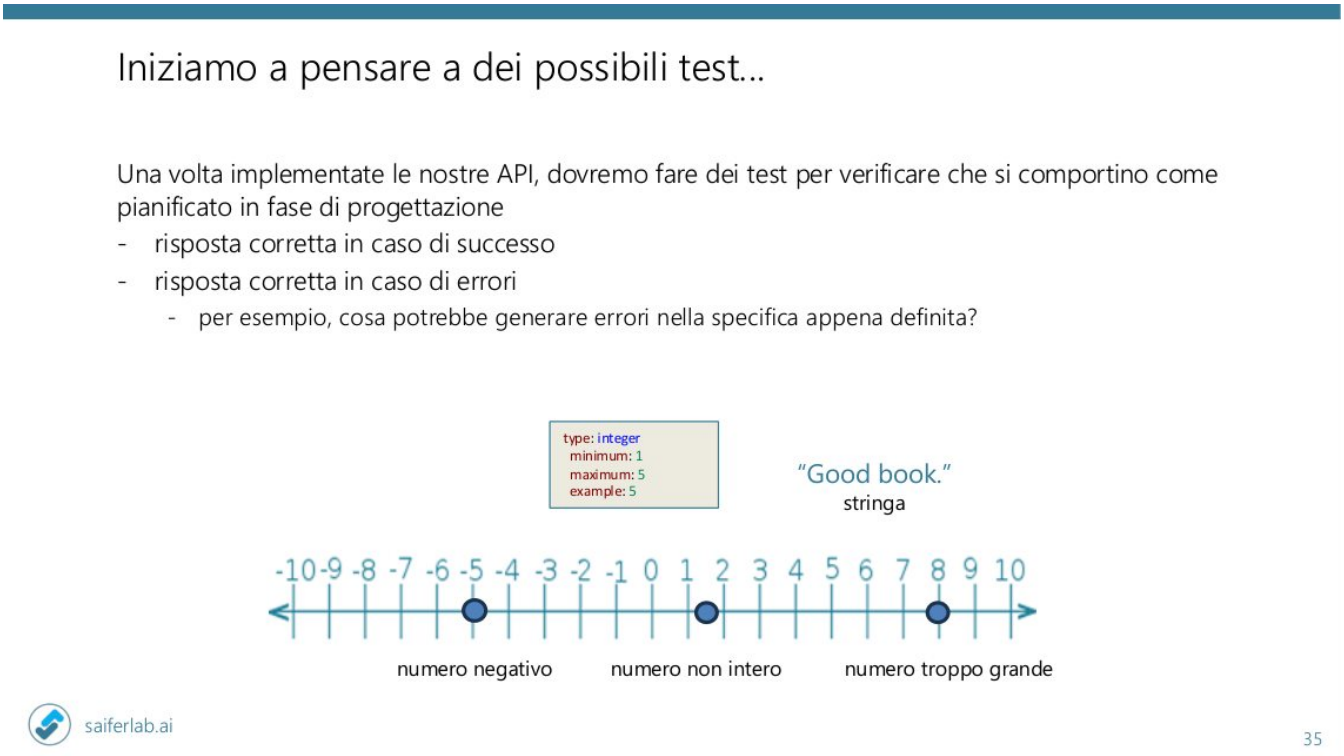
minimum: 1 maximum: 5 example: 5

“Good book.”

stringa

numero negativo numero non intero numero troppo grande

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 036 Componenti di un servizio web

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 36 di 46

Spiegazione per studiare

Un servizio Web può essere organizzato in livelli con responsabilità diverse. La scomposizione rende possibile separare gestione HTTP, regole applicative e persistenza. Il vantaggio è mantenere una struttura più leggibile e consentire modifiche localizzate.

Definizioni, passaggi ed esempi presenti nella slide

Divideremo concettualmente gli elementi che compongono un sito isolando tre strati (layers)

- Livello web
- Livello della logica di servizio
- Livello dei dati

frontend

spesso raggruppati col termine backend

web layer

E aggiungeremo i componenti aggiuntivi

- Modelli e definizioni dei dati (sarà oggetto di lezioni successive)
- Test (sarà oggetto di lezioni successive)

service layer

data layer

Come prima cosa, andiamo a vedere cosa si trova nei diversi livelli

SLIDE ORIGINALE · diagrammi, tabelle e codice

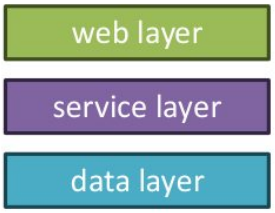
Componenti di un servizio web

Divideremo concettualmente gli elementi che compongono un sito isolando tre strati (layers)

- Livello web
- Livello della logica di servizio
- Livello dei dati

} frontend

} spesso raggruppati col termine **backend**



E aggiungeremo i componenti aggiuntivi

- Modelli e definizioni dei dati (sarà oggetto di lezioni successive)
- Test (sarà oggetto di lezioni successive)

Come prima cosa, andiamo a vedere cosa si trova nei diversi livelli

06 / SLIDE 037 Livello Web

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 37 di 46

Spiegazione per studiare

Il livello Web riceve le richieste, interpreta indirizzi e parametri e costruisce le risposte. Deve gestire l'interfaccia verso il client senza contenere indiscriminatamente tutta la logica dell'applicazione.

Definizioni, passaggi ed esempi presenti nella slide

Il livello concettualmente più alto, quello del Web, fa da interfaccia tra l’utente e i livelli di servizio e di dati

```
Definisce cosa l’utente può fare, di solito in termini di operazioni
CRUD (Create, Read, Update, Delete), e spesso prevede
un’interfaccia utente interattiva
```

web layer

service layer

L’utente comunica con il web layer tramite applicazioni dedicate (client) o API (nei casi di utenti più esperti)

data layer

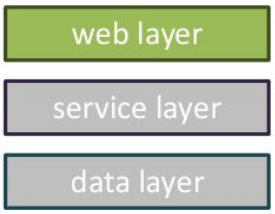
SLIDE ORIGINALE · diagrammi, tabelle e codice

Livello Web

Il livello concettualmente più alto, quello del Web, fa da interfaccia tra l’utente e i livelli di servizio e di dati

Definisce cosa l’utente può fare, di solito in termini di operazioni CRUD (Create, Read, Update, Delete), e spesso prevede un’**interfaccia utente interattiva**

L’utente comunica con il web layer tramite applicazioni dedicate (client) o API (nei casi di utenti più esperti)



saiferlab.ai

37

06 / SLIDE 038 Livello della logica di servizio

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 38 di 46

Spiegazione per studiare

Il livello della logica di servizio realizza le operazioni applicative richieste dal client. Coordina le regole del dominio e, quando necessario, l'accesso alle informazioni persistenti, mantenendo distinti i dettagli del protocollo HTTP.

Definizioni, passaggi ed esempi presenti nella slide

Il livello di servizio contiene quella che viene chiamata anche business logic, ovvero tutti i dettagli di ciò che il servizio web fornisce agli utenti

Il livello di servizio controlla anche come gli utenti possono accedere ai dati, contenuti e gestiti nel layer sottostante

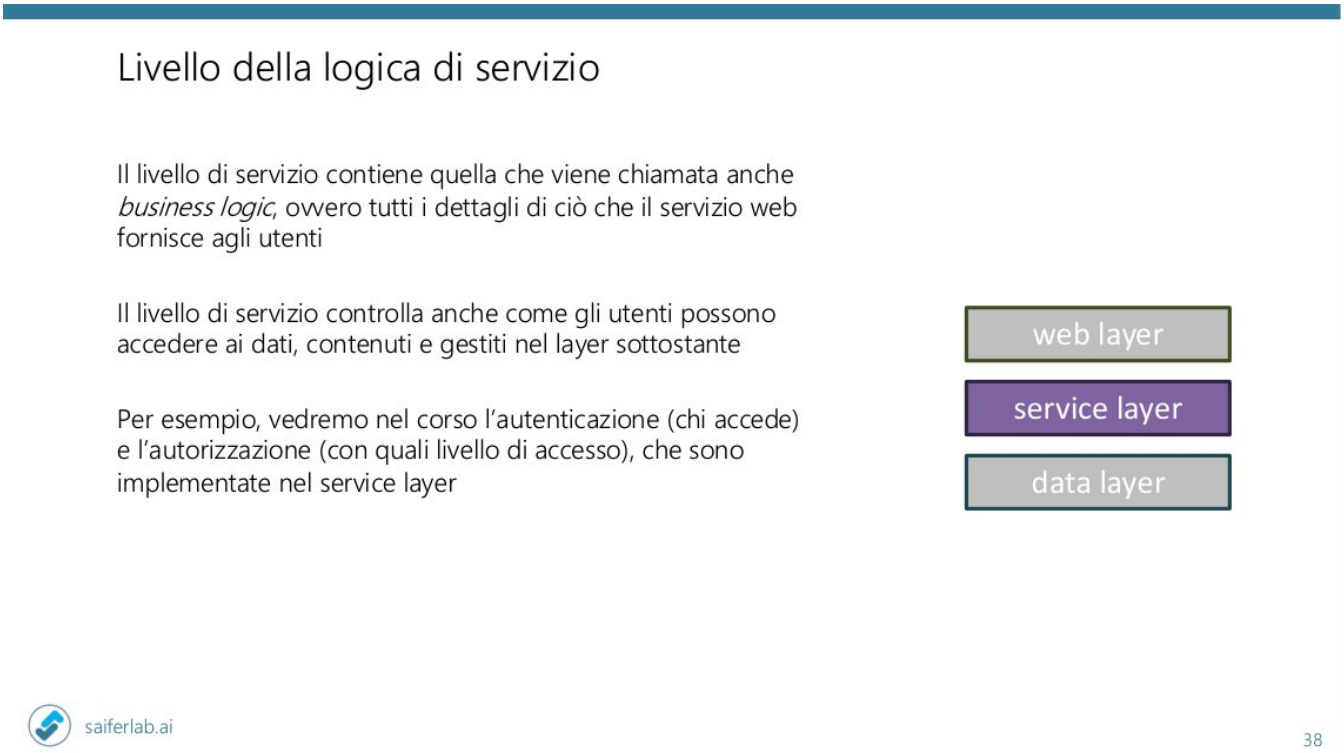
web layer

service layer

Per esempio, vedremo nel corso l'autenticazione (chi accede) e l'autorizzazione (con quali livello di accesso), che sono implementate nel service layer

data layer

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 039 Livello dei dati

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 39 di 46

Spiegazione per studiare

Il livello dei dati gestisce memorizzazione e recupero delle informazioni necessarie all'applicazione. L'accesso ai dati deve essere presentato agli altri livelli tramite interfacce definite, per ridurre l'accoppiamento con un particolare database.

Definizioni, passaggi ed esempi presenti nella slide

Il livello dei dati fornisce al livello di servizio accesso ai dati, attraverso file o funzioni di accesso e modifica di altri servizi che forniscono accesso ai dati (per esempio, servizi di database)

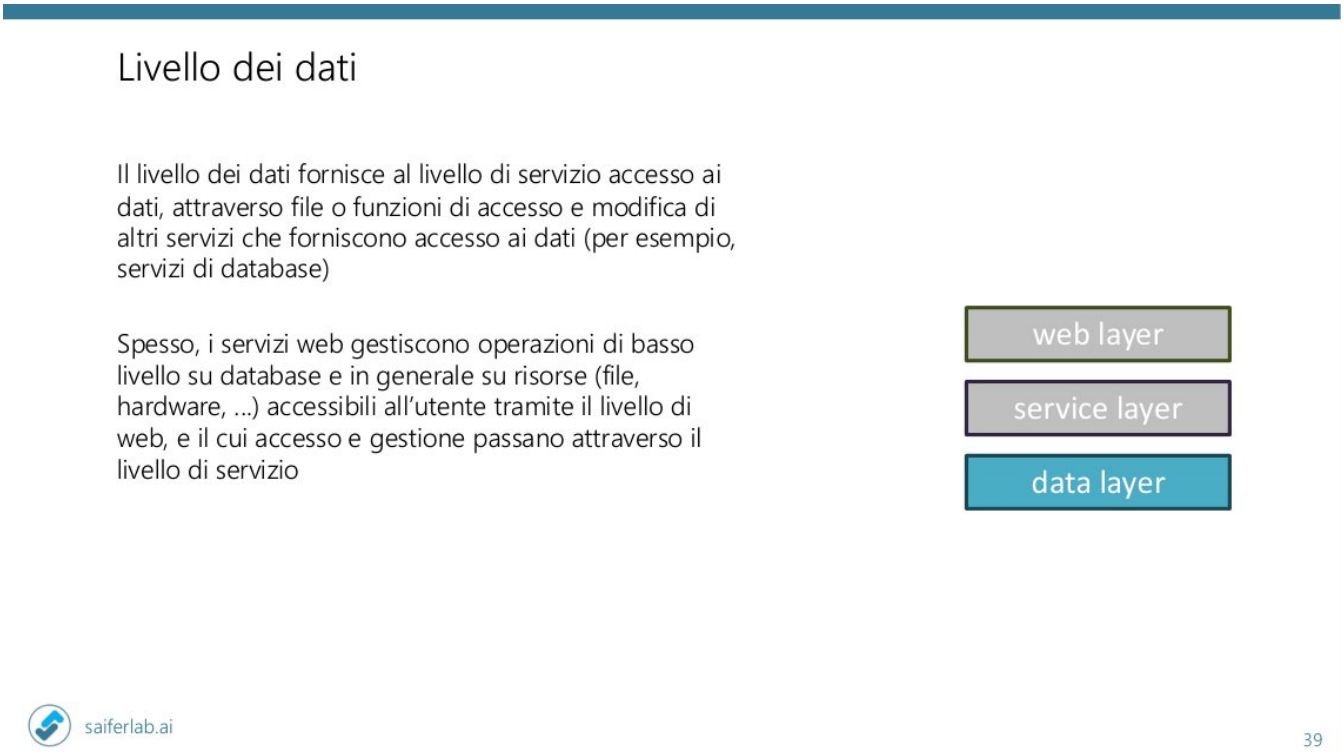
Spesso, i servizi web gestiscono operazioni di basso livello su database e in generale su risorse (file, hardware, ...) accessibili all'utente tramite il livello di web, e il cui accesso e gestione passano attraverso il livello di servizio

web layer

service layer

data layer

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 040 Esempio di gestione di una richiesta

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 40 di 46

Spiegazione per studiare

La slide riunisce i livelli descritti in un ciclo di richiesta: il livello Web riceve la chiamata, la logica di servizio decide che cosa fare e il livello dati gestisce l'eventuale persistenza. Il risultato risale quindi fino alla risposta restituita al client.

Definizioni, passaggi ed esempi presenti nella slide

Per esempio, torniamo alla richiesta della lista di libri vista in precedenza

L’utente richiede la lista di libri tramite il livello Web (per esempio con il browser, o tramite applicazioni

Il web layer mostra i dati all’utente

web layer

specifiche)

Il service layer processa i dati richiesti

il service layer processa i dati e

service layer

dall’utente

li prepara per l’utente

Il data layer recupera i dati

il data layer passa i dati al

data layer

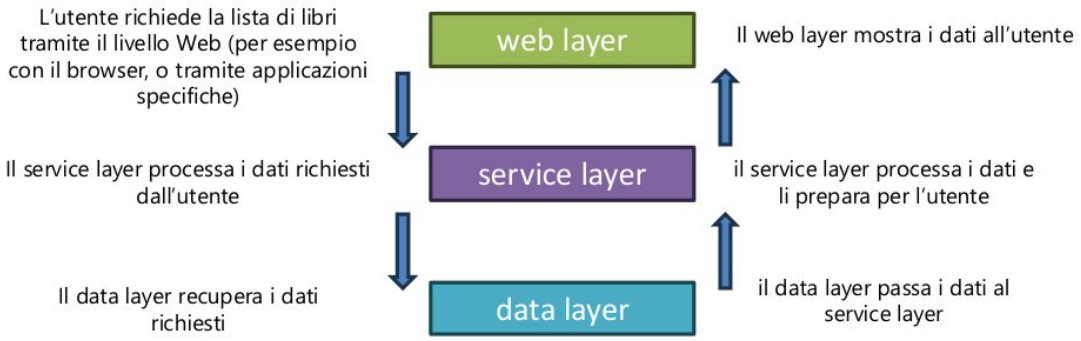
service layer

richiesti

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di gestione di una richiesta

Per esempio, torniamo alla richiesta della lista di libri vista in precedenza



saiferlab.ai

40

06 / SLIDE 041 A cosa serve definire una struttura a strati e avere modularità?

Lezione: Progettazione del Web dinamico e delle API
Fonte: 06_progettazione_web_dinamico.pdf, pagina 41 di 46

Spiegazione per studiare

La modularità separa responsabilità e rende più semplici manutenzione, sviluppo collaborativo e sostituzione di componenti. L'architettura a livelli riduce l'effetto a catena delle modifiche, purché siano definite interfacce coerenti tra i moduli.

Definizioni, passaggi ed esempi presenti nella slide

Ci sono diversi vantaggi derivanti dall’isolamento concettuale dei diversi livelli

• ogni layer può essere progettato da diversi specialisti

• si possono eseguire test in isolamento

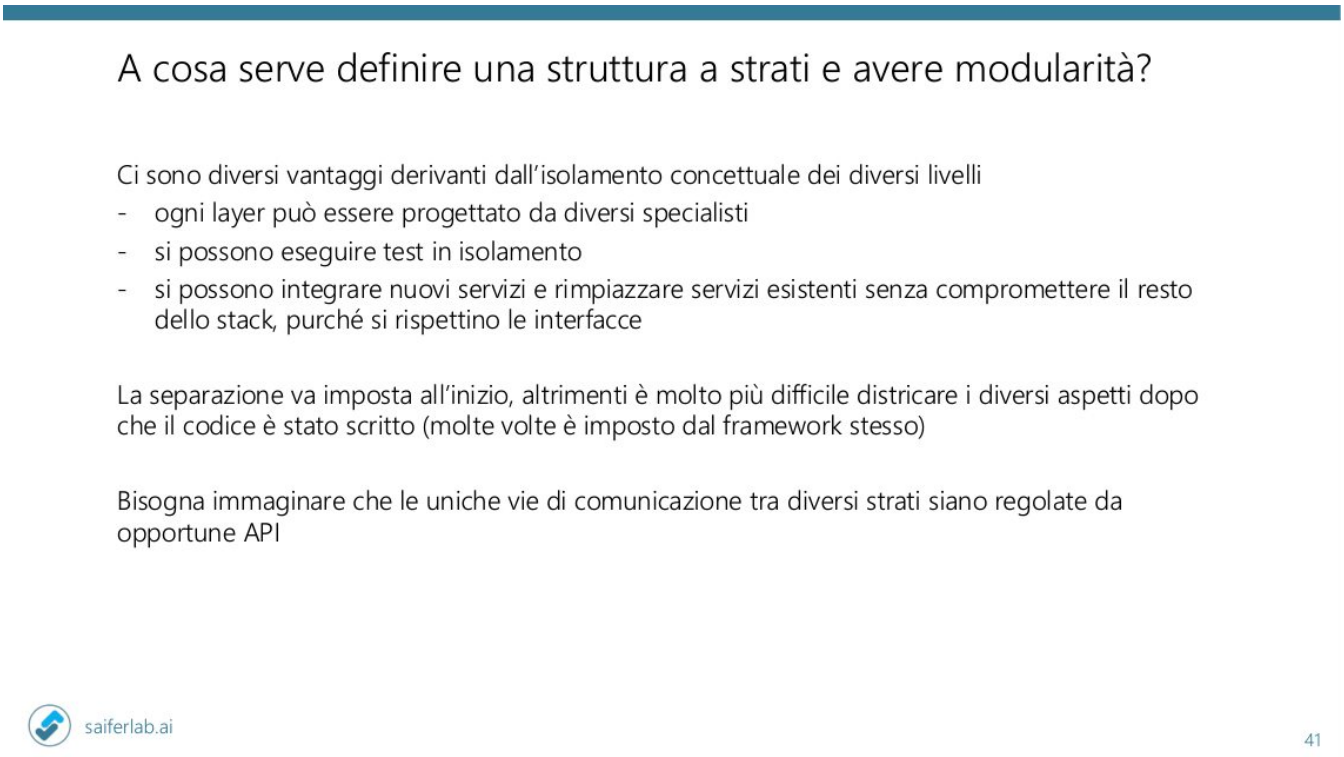
• si possono integrare nuovi servizi e rimpiazzare servizi esistenti senza compromettere il resto

dello stack, purché si rispettino le interfacce

La separazione va imposta all’inizio, altrimenti è molto più difficile districare i diversi aspetti dopo che il codice è stato scritto (molte volte è imposto dal framework stesso)

Bisogna immaginare che le uniche vie di comunicazione tra diversi strati siano regolate da opportune API

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 042 Architetture a microservizi

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 42 di 46

Spiegazione per studiare

I microservizi scompongono un'applicazione in servizi separati che comunicano fra loro tramite interfacce. Questa scelta consente una suddivisione più netta delle funzionalità, ma introduce la necessità di gestire comunicazioni e coordinamento tra parti distinte.

Definizioni, passaggi ed esempi presenti nella slide

I microservizi sono semplicemente dei servizi indipendenti, autonomi, e compatti, che collaborano

• sono applicazioni che girano in modo indipendente

• comunicano tramite API

• l'idea di essere "compatti" non deve essere confusa

con l'essere "piccoli", ma che sono applicazioni con uno scopo preciso e verticale

• principio di "Single Responsiblity", ovvero essere

responsabilit soltanto di un compito, ma farlo bene

• per esempio, inviare mail, invio di pagamenti,

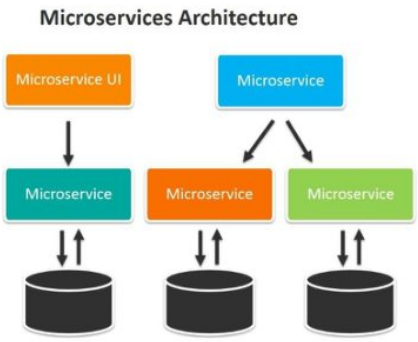
trasferimento dei file, possono essere implementati come microservizi isolati

SLIDE ORIGINALE · diagrammi, tabelle e codice

Architetture a microservizi

I microservizi sono semplicemente dei servizi indipendenti, autonomi, e compatti, che collaborano

- sono applicazioni che girano in modo indipendente
- comunicano tramite API
- l'idea di essere "compatti" non deve essere confusa con l'essere "piccoli", ma che sono applicazioni con uno scopo preciso e verticale
  - principio di "Single Responsiblity", ovvero essere responsabilit soltanto di un compito, ma farlo bene
  - per esempio, inviare mail, invio di pagamenti, trasferimento dei file, possono essere implementati come microservizi isolati



saiferlab.ai

42

06 / SLIDE 043 Architetture a microservizi

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 43 di 46

Spiegazione per studiare

La seconda slide sui microservizi completa l'analisi, mettendo in evidenza proprietà e conseguenze operative della scomposizione. Per valutarli, bisogna considerare non solo la modularità ma anche la complessità delle connessioni e della distribuzione.

Definizioni, passaggi ed esempi presenti nella slide

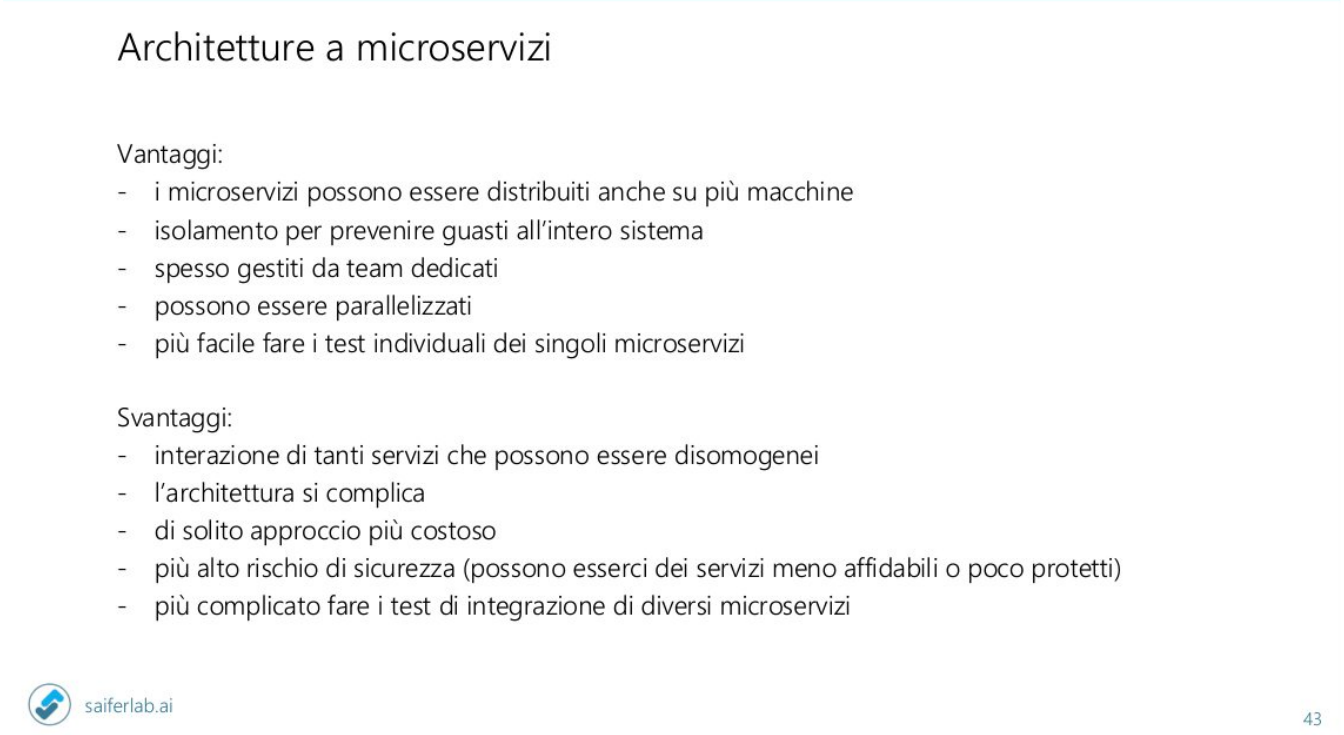
Vantaggi:

- i microservizi possono essere distribuiti anche su più macchine
- isolamento per prevenire guasti all'intero sistema
- spesso gestiti da team dedicati
- possono essere parallelizzati
- più facile fare i test individuali dei singoli microservizi

Svantaggi:

- interazione di tanti servizi che possono essere disomogenei
- l'architettura si complica
- di solito approccio più costoso
- più alto rischio di sicurezza (possono esserci dei servizi meno affidabili o poco protetti)
- più complicato fare i test di integrazione di diversi microservizi

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 044 Le architetture monolitiche

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 44 di 46

Spiegazione per studiare

In un'architettura monolitica le funzionalità dell'applicazione sono organizzate in un unico sistema da costruire e distribuire. Questo modello concentra la gestione del software, pur richiedendo attenzione quando il numero delle funzionalità cresce.

Definizioni, passaggi ed esempi presenti nella slide

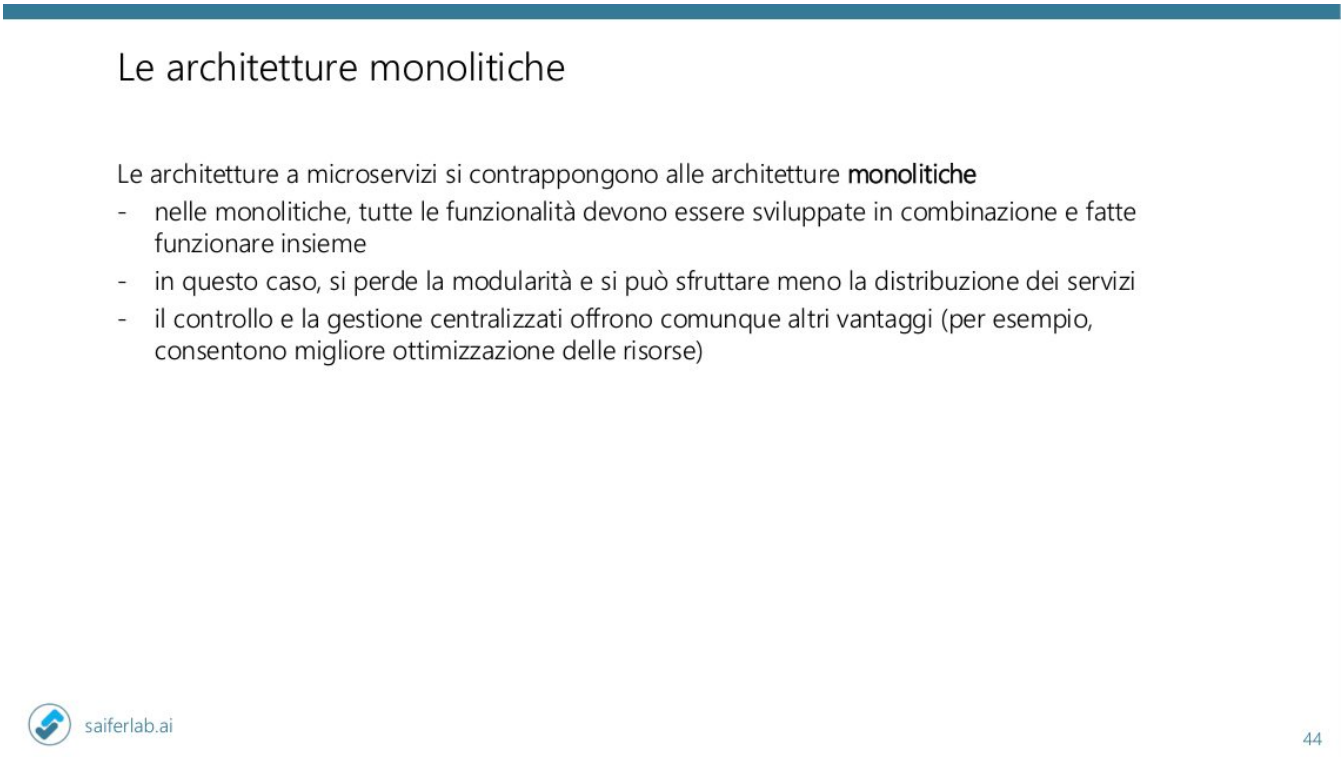
Le architetture a microservizi si contrappongono alle architetture monolitiche

• nelle monolitiche, tutte le funzionalità devono essere sviluppate in combinazione e fatte funzionare insieme

• in questo caso, si perde la modularità e si può sfruttare meno la distribuzione dei servizi

• il controllo e la gestione centralizzati offrono comunque altri vantaggi (per esempio, consentono migliore ottimizzazione delle risorse)

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 045 Architetture monolitiche

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 45 di 46

Spiegazione per studiare

La slide prosegue il confronto sui monoliti: la coesione dell'applicazione può semplificare l'avvio e la distribuzione, ma può rendere meno indipendenti le modifiche alle diverse parti del sistema. Il trade-off va compreso rispetto alla scala del progetto.

Definizioni, passaggi ed esempi presenti nella slide

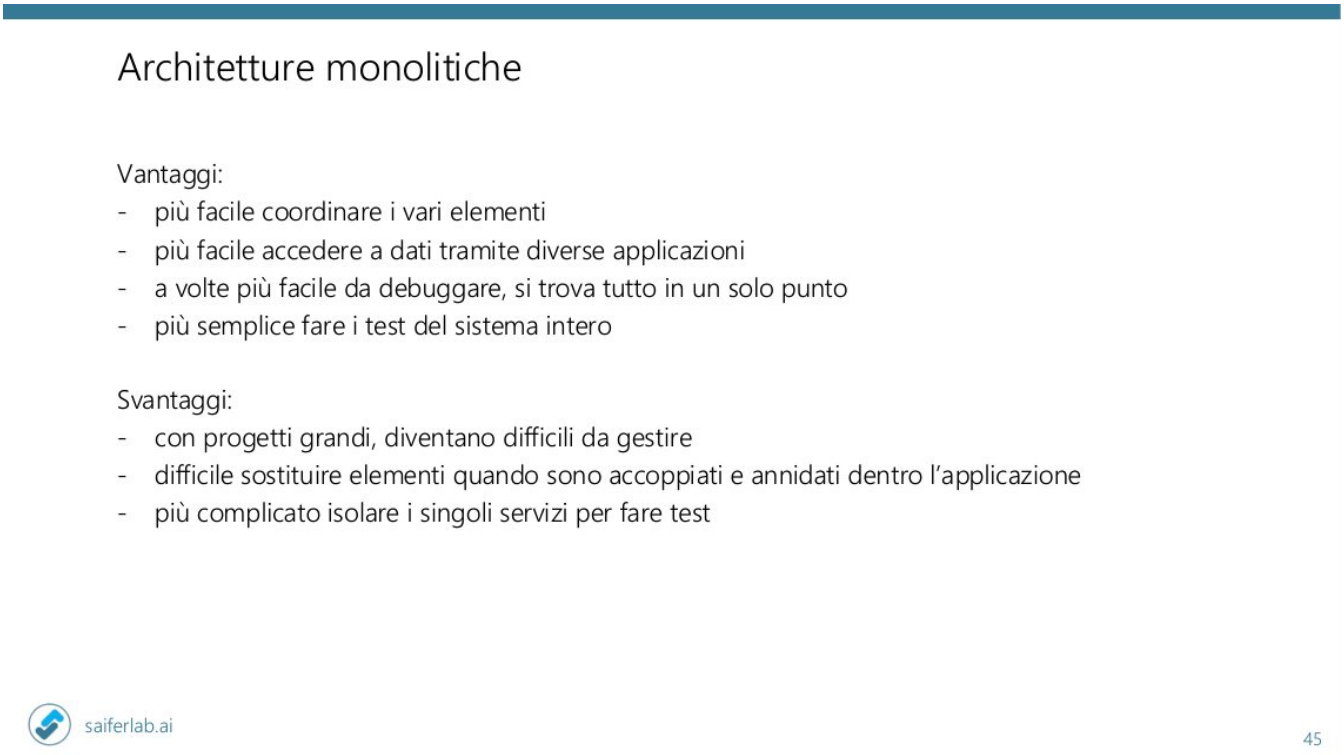
Vantaggi:

- più facile coordinare i vari elementi
- più facile accedere a dati tramite diverse applicazioni
- a volte più facile da debuggare, si trova tutto in un solo punto
- più semplice fare i test del sistema intero

Svantaggi:

- con progetti grandi, diventano difficili da gestire
- difficile sostituire elementi quando sono accoppiati e annidati dentro l'applicazione
- più complicato isolare i singoli servizi per fare test

SLIDE ORIGINALE · diagrammi, tabelle e codice



06 / SLIDE 046 Quindi, quale scegliere?

Lezione: Progettazione del Web dinamico e delle API

Fonte: 06_progettazione_web_dinamico.pdf, pagina 46 di 46

Spiegazione per studiare

La scelta tra monolite e microservizi non è assoluta: dipende dalla dimensione dell'applicazione, dalle esigenze di sviluppo e dalla complessità che il gruppo può gestire. Bisogna giustificare la decisione in base ai requisiti, non trattare una delle due architetture come sempre superiore.

Definizioni, passaggi ed esempi presenti nella slide

In generale, la scelta è data dal framework e dai servizi che si vogliono usare

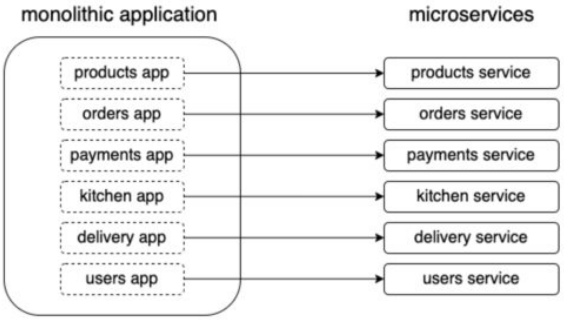
- molti framework (per esempio Django in Python) sono pensati per applicazioni monolitiche
- di solito si scelgono vie di mezzo (alcuni servizi sono gestiti come microservizi, altri sono integrati nell'applicazione)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Quindi, quale scegliere?

In generale, la scelta è data dal framework e dai servizi che si vogliono usare

- molti framework (per esempio Django in Python) sono pensati per applicazioni monolitiche
- di solito si scelgono vie di mezzo (alcuni servizi sono gestiti come microservizi, altri sono integrati nell'applicazione)



saiferlab.ai

46